2026년 11월 1일 08:50

1. **들어가며.** 이 프로그램은 **비스듬한 삼진 나무**(skew ternary tree)를 가지고 셈을 하고, 그것에 대응하는 **나눌 수 없는 평면 그래프**를 보여 준다. 커누스가 2013년 11월에 찾아낸 몇 가지 간단한 알고리즘을 담았는데, 그 바탕은 Alberto Del Lungo와 Francesco Del Ristoro와 Jean-Guy Penaud의 생각이다(*Theoretical Computer Science* **233** (2000), 201–215). 그는 이 놀라운 대응이 지닌, 마술 같아 보이는 성질을 더 알고 싶어서 이것을 지었다고 적었다.

커누스는 더 나은 사용자 인터페이스를 만들 틈도, 더 넉넉한 주석을 달 틈도 없었다고 사과해 두었다. 나무와 그래프를 손으로 만지작거릴 수 있는 대화식 프로그램이 있으면 좋겠고, 거기에 숨은 매혹적인 무늬들을 색으로 드러내면 더 좋겠다고. 누군가 그런 "앱"을 만들 마음이 들기를 바란다면서, 그것이 틀림없이 굉장히 가르침이 많은 장난감이 되리라고 적었다.

나는 그 기대에 부응해 보려고 이 저장소의 `app` 디렉터리에 그런 앱을 만들어 `sjnam.github.io/skew-ternary`에 올려 두었다(2026년 11월). 브라우저만 있으면 도는 한 장짜리 페이지 `app/index.html`이고, 판이 넷이다. 첫째 판은 아래 그림 1처럼 싹까지 드러낸 나무를 그리는데, 세로축을 깊이에서 계급으로 바꾸면 0선 아래로 내려가는 마디가 없다는 것, 곧 비스듬함이 지형처럼 드러난다. 둘째 판에서는 개미 앨리스를 한 걸음씩 걸리며 상태표를 따라가므로, 뒤에 나올 정리의 증명이 눈앞에서 그대로 돌아간다. 셋째 판은 켤레 넷을 늘어놓아 누르는 대로 그 나무로 옮겨 가고, 넷째 판은 켤레마다 두 대응의 평면 지도를 그려 꼭짓점 고리와 면 고리를 함께 보이며 쌍대를 겹쳐 볼 수 있게 한다.

셈하는 부분은 이 문서의 「새기기」와 「상태표」 두 장, 그리고 quad-edge 자료 구조부터 Del Lungo 들의 평면 지도까지 네 장을 자바스크립트로 옮긴 것이다. 서로 다른 무작위 나무 8900여 그루에서 이 프로그램의 출력 전체와 글자 하나까지 같았다. 지도의 그림만은 원본에 없다. 프로그램이 내놓는 것은 꼭짓점과 변과 면의 목록뿐이라 그것은 옮기기가 아니라 짓기였다. 커누스가 바란 만큼 훌륭한 장난감인지는 모르겠으나, 적어도 그것을 만들며 가르침을 가장 많이 받은 사람은 나다.

2. 적어도 기본은 여기서 밝혀 두겠다. **삼진 나무**는 비었거나, 아니면 뿌리 마디 하나와 삼진 나무 셋으로 이루어진다. 그 셋을 뿌리의 왼쪽, 가운데, 오른쪽 부분나무라 하고, 그 뿌리들을 뿌리 마디의 왼쪽, 가운데, 오른쪽 자식이라 한다. (이것은 커누스의 책 *Fundamental Algorithms* 2.3절에 나오는 이진 나무의 정의를 그대로 본뜬 것이다.)

여기에 더해, 빈 부분나무가 있던 자리마다 **싹**(bud)을 놓아 나무를 늘린다. 꼭대기에도 싹을 하나 두어 뿌리 마디에 붙인다. 그러면 뿌리를 포함한 모든 마디가 정확히 네 개의 다른 마디나 싹에 붙고, 모든 싹은 정확히 하나의 마디나 싹에 붙는다. 구조를 자세히 들여다보려고 마디마다 싹마다 이름을 붙여 둔다.

마디가 n개인 늘린 삼진 나무에는 언제나 싹이 $2n+2$개 있다. (n에 대한 귀납법으로 쉽게 밝혀지는 이 사실은 삼진 나무가 비었을 때도 성립한다. 그때는 $n=0$이고 서로 붙은 싹 둘만 있다.)

그런 나무를 평면에 앉히면 "고리처럼 같은" 늘린 삼진 나무 $2n+2$개의 무리가 생긴다. 싹 하나마다 나무가 하나씩 나오는데, 그 싹을 꼭대기에 두고 나머지를 모두 거기 매달면 된다. 이 나무들은 저마다 뿌리 싹이 다르지만 뿌리 마디까지 다른 것은 아니다. 서로 다른 싹이 같은 뿌리 마디를 가진 다른 나무로 이어질 수 있기 때문이다.

싹 $2n+2$개는 언제나 둘씩 짝지어 $n+1$개의 무리로 나뉜다. 어떤 싹의 짝을 찾으려면 그 싹에서 출발해 오직 하나뿐인 길을 따라가면 된다. 갈림길이 셋일 때는 언제나 가운데 가지를 골라 가다가 다른 싹을 만나면 거기가 짝이다.

마디와 싹에는 다음처럼 자연스럽게 **계급**(rank)을 매긴다. 뿌리 마디와 뿌리 싹의 계급은 0이고, 계급이 r인 마디의 왼쪽, 가운데, 오른쪽 자식의 계급은 각각 $r-1$, r, $r+1$이다.

비스듬한 삼진 나무란 모든 마디의 계급이 음이 아닌 삼진 나무를 말한다.

3. 보기를 들어 아래 비스듬한 삼진 나무는 마디가 6개, 싹 짝이 7개다. 계급은 빨간색으로 적었다. 계급이 0인 마디마다 계급이 −1인 싹이 하나씩 있음을 눈여겨보라.

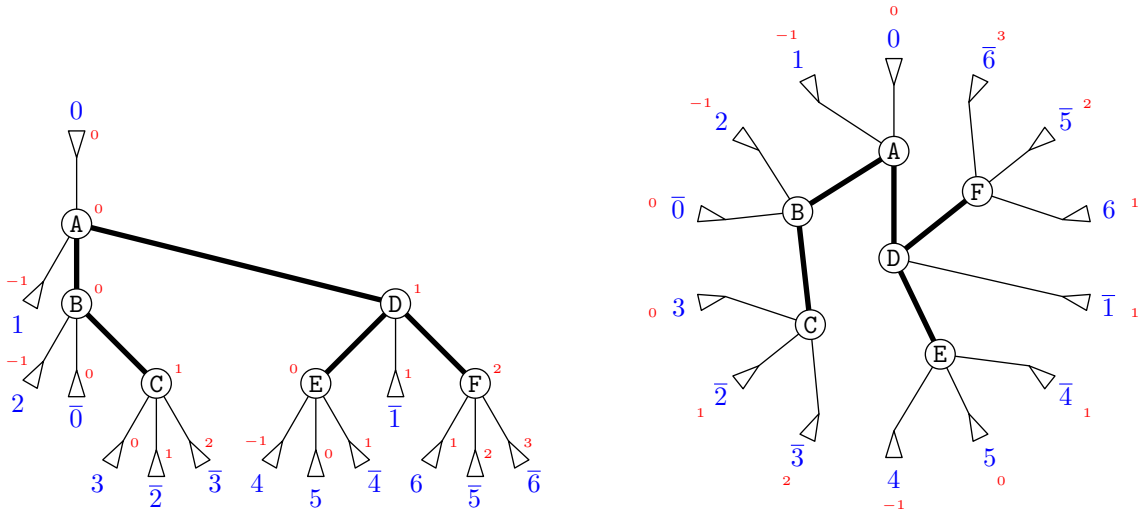


그림 1: 마디 여섯인 비스듬한 삼진 나무. 왼쪽은 평면에 매달아 그린 것이고, 오른쪽은 같은 나무를 고리 모양으로 펼친 것이다. 싹에 붙은 파란 번호가 싹 이름이고, 윗줄이 그어진 것은 그 짝이다.

4. 사실 하나: 고리처럼 같은 삼진 나무 $2n+2$개로 이루어진 무리마다, 비스듬한 삼진 나무가 정확히 넷 들어 있다.

이 프로그램이 존재하는 가장 큰 까닭인 이 정리에는 놀랍도록 간단한 증명이 있다. 삼진 나무의 마디와 마디를 잇는 변 $n-1$개를 생각하고, 변 U—V마다 그것을 호 U⟶V와 V⟶U의 짝으로 다루자. 그러면 호가 $2n-2$개 생긴다. 그리고 아래 그림처럼 서로 엇갈리지 않는 실 $2n-2$가닥으로 그 호들을 짝 $2n+2$개 가운데 $2n-2$개에 자연스럽게 맞붙일 수 있다.

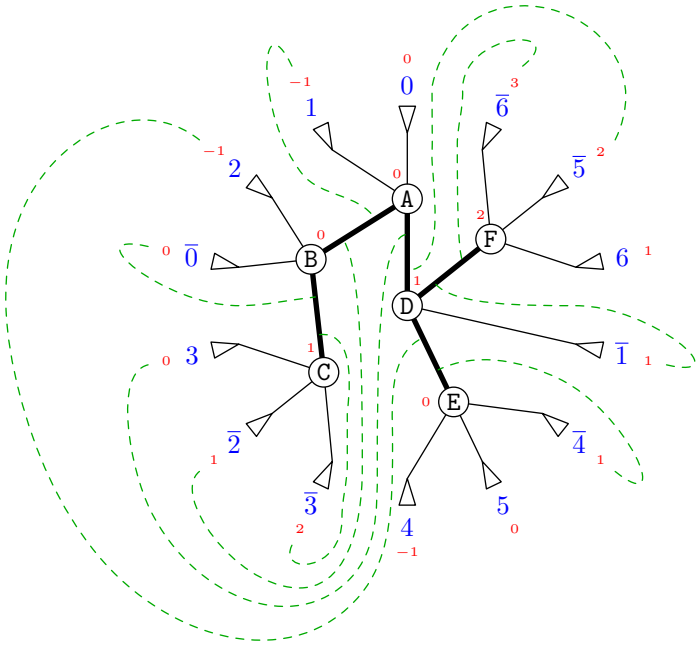


그림 2: 같은 나무에 실을 걸어 호와 싹을 맞붙인 모습. 초록 점선이 실이고, 맞붙지 못한 싹 넷이 비스듬한 나무 넷에 해당한다.

5. 좀 더 또렷이 말하자면, 앨리스라는 개미가 나무의 둘레를 기어간다고 하자. 앨리스는 0번 싹의 바로 오른쪽에서 상태 -2로 출발한다. 싹을 지날 때마다 상태를 1 올리고, 호를 지날 때마다 1 내린다. 그러면 제자리로 돌아왔을 때 상태가 $-2+(2n+2)-(2n-2)=+2$가 된다.

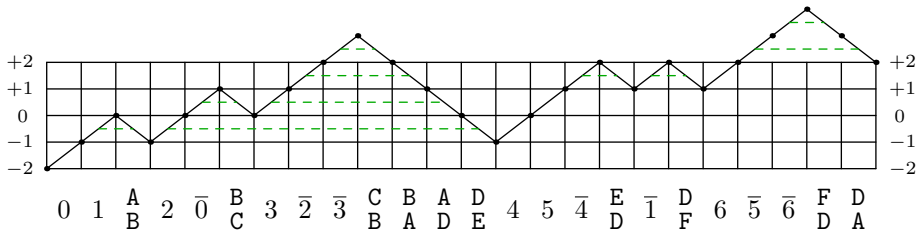


그림 3: 앨리스가 0번 싹에서 출발해 한 바퀴 도는 동안의 상태. 초록 점선은 맞붙은 호와 싹을 잇는다.

계속 기어가면 같은 무늬가 되풀이되는데, 다만 상태가 4씩 올라간다.

6. 요점은 이것이다. 앨리스가 계급이 k인 싹에 이를 때 그의 상태는 언제나 k다. 출발한 싹만 예외로 ± 2다. 그러니 맞붙지 못한 싹들이 곧 비스듬한 삼진 나무에 해당한다. 이 보기에서는 0번, 4번, 5번, 6번 싹에 매달린 나무에 계급이 음인 마디가 없다. 거꾸로, 맞붙은 싹에서 시작하는 삼진 나무에는 계급이 -1보다 작은 싹이 적어도 하나 있고, 따라서 계급이 0보다 작은 마디가 적어도 하나 있다.

QED.

(이 증명을 이해한 독자라면 고리처럼 같은 **오진 나무** $4n+2$개의 무리마다 비스듬한 오진 나무가 정확히 여섯 들어 있다는 것도 밝힐 수 있을 것이다. 그다음도 마찬가지다.)

7. 한 고리 무리에 든 비스듬한 삼진 나무 넷은 놀라운 성질을 지닌다. 맞붙지 못한 싹 넷에서 저마다 출발할 때 앨리스가 겪는 상태 변화를 보자.

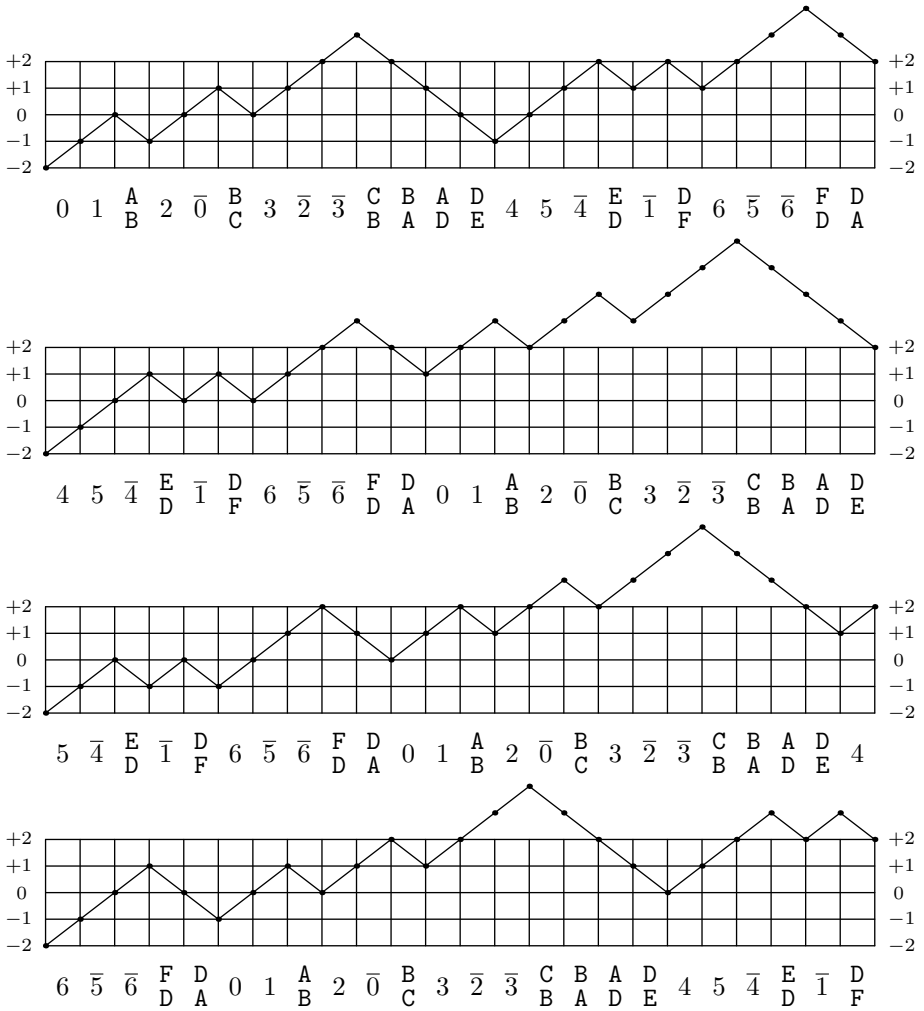
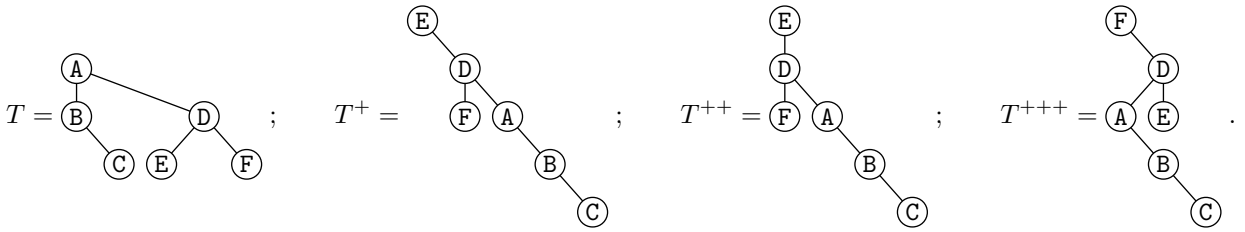


그림 4: 맞붙지 못한 네 싹에서 각각 출발한 상태표. 위에서부터 0번, 4번, 5번, 6번 싹이다.

8. 거기에 대응하는 비스듬한 삼진 나무들은, 싹을 빼고 그리면 이렇다.



여기 쓴 표기법을 눈여겨보라. 비스듬한 삼진 나무 하나를 다른 것으로 보내는, 잘 정의된 연산 $T \mapsto T^{+}$에 바탕을 둔 것이다. $T^{++++} = T$이므로 T^{+++}를 T^{-}로 줄여 쓰기도 하고, T^{++}는 T^{--}라 불러도 된다.

9. 이 프로그램의 첫 목표는 비스듬한 삼진 나무 T가 주어졌을 때 그 "켤레" T^{+}, T^{++}, $T^{+++} = T^{-}$를 셈하는 것이다. 나무는 명령줄로 준다. 네 글자짜리 인자 `abcd`를 늘어놓는데, 첫 글자 `a`는 마디의 이름이고 이어지는 세 글자는 그 마디의 세 자식이다. 빈 자식은 '`-`'로 적는다. 보기를 들어 위의 나무 T는 이 여섯 인자로 적을 수 있다(차례는 아무래도 좋다).

```
A-BD    B--C    C---    DE-F    E---    F---
```

마디마다 인자가 하나씩 있어야 한다. 프로그램은 인자를 새기면서 그것이 정말로 비스듬한 삼진 나무를 이루는지 살핀다.

10. 이것은 커누스의 `CWEB` 프로그램 `skew-ternary-calc.w`를 `GWEB`으로 옮긴 것이다. 원본의 머리글 `Last-Modified`는 `Sun, 30 Jul 2017 07:01:22 GMT`다. 그림은 커누스가 함께 내놓은 `skew-ternary-calc.mp`의 것을 그대로 쓰되, 조판 중에 luamplib이 그릴 수 있도록 손을 보았다. 그 이야기는 맨 뒤에서 하겠다.

11. 이제 비스듬한 삼진 나무가 무엇인지 알았으니 코드를 짤 때가 되었다. 프로그램의 큰 얼개는 이렇다.

```
package main
import (
    "fmt"
    "os"
    "strings"
)
⟨ 상수 13 ⟩
⟨ 자료형 14 ⟩
⟨ 전역 변수 15 ⟩
⟨ 함수들 17 ⟩
func main() {
    ⟨ 지역 변수 16 ⟩
    ⟨ 명령줄을 처리한다 12 ⟩
    ⟨ T 의 켤레 셋을 찾아 내놓는다 30 ⟩
    ⟨ 대응하는 평면 지도를 찾아 내놓는다 72 ⟩
}
```

12. 빈 나무는 다루지 않는다. 마디가 적어도 하나는 있어야 한다.

⟨ 명령줄을 처리한다 12 ⟩ ≡

```
if len(os.Args) ≡ 1 {
    fmt.Fprintf(os.Stderr, "쓰는␣법:␣%s␣마디 1␣마디 2␣...␣마디 n\n", os.Args[0])
    os.Exit(1)
}
⟨ 인자를 새긴다. 비스듬한 나무가 아니면 알리고 그만둔다 18 ⟩
```

이 코드는 11 절에서 쓰인다.

13. 새기기. 먼저 할 일부터. 나무를 다루기 좋은 꼴로 메모리에 들여놓아야 한다. 바탕이 되는 자료 구조는 마디마다 *left*, *middle*, *right*, *parent* 밭을 지니고, 가다 보면 쓸 것이 몇 개 더 있다. 싹은 음의 정수로 나타내고, 그 밖의 이음줄은 마디의 여덟 비트 글자 코드를 가리킨다.

(이 구현에서 눈에 보이는 글자 코드는 많아야 64가지다. * 하나와 @부터 ~까지 63개다.)

⟨ 상수 13 ⟩ ≡

```
const (
    sentinel = 999
    maxcodes = 64
)
```

이 코드는 32 절에도 정의되어 있다.

이 코드는 11 절에서 쓰인다.

14. ⟨ 자료형 14 ⟩ ≡

```
type node struct {
    left int    // 왼쪽 자식
    middle int    // 가운데 자식
    right int    // 오른쪽 자식
    parent int    // 부모
    rank int    // 읽어 들일 때는 sentinel, 나중에 진짜 계급이 된다
}
type bud struct {
    parent int    // 부모
    rank int    // 계급
    stepno int    // 상태표에서의 걸음 번호(아래를 보라)
}
```

이 코드는 28 절에도 정의되어 있다.

이 코드는 11 절에서 쓰인다.

15. ⟨ 전역 변수 15 ⟩ ≡

```
var (
    inputnode [256]node    // 실제로는 @부터 ~까지만 쓴다
    inputbud [512]bud    // k번 싹의 자료는 inputbud[−k]에 있다
    buds int    // 여태 만든 싹의 수
    n int    // 나무의 마디 수
)
```

이 코드는 29, 50, 66 절에도 정의되어 있다.

이 코드는 11 절에서 쓰인다.

16. ⟨ 지역 변수 16 ⟩ ≡

```
var c, i, j, k, p int
```

이 코드는 11 절에서 쓰인다.

17. 처음 채비는 곧이곧대로지만 좀 지루하다. 인자가 잘못됐을 때 알리고 그만두는 일이 세 가지 꼴로 되풀이되므로 함수로 두었다.

⟨ 함수들 17 ⟩ ≡

```
func abort0(message string, code int) {
    fmt.Fprintf(os.Stderr, "%s!\n", message)
    os.Exit(code)
}
func abort1(message string, j, code int) {
    fmt.Fprintf(os.Stderr, "잘못된␣인자␣(%s):␣%s!\n", os.Args[j], message)
    os.Exit(code)
}
func abort2(message string, j, c, code int) {
    fmt.Fprintf(os.Stderr, "잘못된␣인자␣(%s):␣마디␣'%c'␣--␣%s!\n",
        os.Args[j], c, message)
    os.Exit(code)
}
```

이 코드는 21, 25, 31, 33, 34, 36, 39, 49, 52, 53, 67, 80, 91 절에도 정의되어 있다.

이 코드는 11 절에서 쓰인다.

18. ⟨ 인자를 새긴다. 비스듬한 나무가 아니면 알리고 그만둔다 18 ⟩ ≡

```
for j = 1; j < len(os.Args); j++ {
    arg := os.Args[j]
    if len(arg) ≠ 4 {
        abort1("네␣글자여야␣한다", j, 10)
    }
    c = int(arg[0])
    if c < '@' ∨ c > '~' {
        abort2("쓸␣수␣없는␣글자다", j, c, 15)
    }
    if inputnode[c].rank ≠ 0 {
        abort2("이미␣정해진␣마디다", j, c, 11)
    }
    inputnode[c].rank = sentinel
    ⟨ 자식 셋을 붙인다 19 ⟩
}
n = len(os.Args) − 1
⟨ 싹을 넣고 계급을 셈한다 20 ⟩
```

이 코드는 12 절에서 쓰인다.

19. 왼쪽, 가운데, 오른쪽이 똑같은 모양이라 고리 하나로 묶었다. 커누스는 셋을 펼쳐 적었는데, 그러면 오류 부호가 저마다 달라져 어느 자식에서 걸렸는지 알 수 있다. 여기서도 그것을 살려 두려고 자식 번호를 부호에 얹었다.

```
⟨ 자식 셋을 붙인다 19 ⟩ ≡
    for i = 1; i ≤ 3; i++ {
        p = int(arg[i])
        if p ≡ '-' {
            continue
        }
        if p < '@' ∨ p > '~' {
            abort2("쓸␣수␣없는␣글자다", j, p, 15 + i)
        }
        if inputnode[p].parent ≠ 0 {
            abort2("이미␣부모가␣있다", j, p, 11 + i)
        }
        inputnode[p].parent = c
        switch i {
        case 1:
            inputnode[c].left = p
        case 2:
            inputnode[c].middle = p
        default:
            inputnode[c].right = p
        }
    }
```

이 코드는 18 절에서 쓰인다.

20. 뿌리를 찾아야 한다. 부모가 없는 입력 마디가 꼭 하나 있어야 하고, 그것이 뿌리다. 거기에 -1번 싹을 붙인다. 싹 $-2k-1$과 $-2k-2$는 짝이다. 그러니 싹 x의 짝은 싹 $x \oplus 1$이다.

커누스는 뿌리를 $inputbud[1].parent$에 그대로 얹어 두고 *root* 라는 이름을 붙였다. 뿌리 싹(1번)의 부모가 곧 뿌리이기 때문이다. Go에는 그런 별명이 없으므로 그 자리를 그대로 쓴다.

⟨ 싹을 넣고 계급을 셈한다 20 ⟩ ≡

```
for j = '@'; j ≤ '~'; j++ {
    if inputnode[j].rank ≠ 0 ∧ inputnode[j].parent ≡ 0 {
        if inputbud[1].parent ≠ 0 {
            fmt.Fprintf(os.Stderr, "뿌리가 둘이다: '%c', '%c'!\n",
                inputbud[1].parent, j)
            os.Exit(20)
        }
        inputbud[1].parent = j
    }
    if inputnode[j].parent ≠ 0 ∧ inputnode[j].rank ≡ 0 {
        fmt.Fprintf(os.Stderr, "마디 '%c'에 대한 자료가 없다!\n", j)
        os.Exit(21)
    }
}
if inputbud[1].parent ≡ 0 {
    abort0("뿌리가 없다", 21)
}
inputbud[1].rank = −2   // 1번 싹은 뿌리 마디 위의 "뿌리 싹"이다
setmate(inputbud[1].parent)   // 그 짝을 찾아 정한다
fillbuds(inputbud[1].parent, 0)
⟨ 나무를 온전히 채웠는지 살핀다 27 ⟩
```

이 코드는 18절에서 쓰인다.

21. 함수 *setmate*는 새 싹 둘을 마련한다. 매개변수는 그런 싹이 있음을 알아챈 마디의 이름이다.

거의 모든 경우에 짝은 가운데 이음줄을 따라 위로 올라갔다가, 왼쪽에서 오른쪽으로 (또는 그 반대로) 건너뛴 다음, 가운데 이음줄을 따라 내려가면 나온다.

⟨ 함수들 17 ⟩ +≡

```
func setmate(p int) {
    var q, d int
    ⟨ 싹 둘을 새로 마련한다 22 ⟩
    ⟨ 가운데를 따라 위로 올라가 옆으로 건너뛴다 23 ⟩
    ⟨ 가운데를 따라 내려가 짝을 놓는다 24 ⟩
}
```

22. 뿌리 싹(1번)만은 짝의 부모가 이미 정해져 있다. 그때는 올라갈 것 없이 곧바로 내려간다.

⟨ 싹 둘을 새로 마련한다 22 ⟩ ≡

```
buds += 2
q = 1 − buds
if inputbud[buds − 1].parent ≠ 0 {
    if buds > 2 {
        confusion("bud parent already set")
    }
    goto downwardmid
}
inputbud[buds − 1].parent = p
```

이 코드는 21 절에서 쓰인다.

23. 지금 자리 p에서 방금 온 곳 q가 가운데 자식이면 더 올라간다. 왼쪽 자식이었으면 오른쪽으로, 오른쪽 자식이었으면 왼쪽으로 건너뛰고 내려갈 채비를 한다. d가 어느 쪽으로 건너뛰었는지 적어 둔다.

⟨ 가운데를 따라 위로 올라가 옆으로 건너뛴다 23 ⟩ ≡

```
upward:
if inputnode[p].middle ≡ q {
    q, p = p, inputnode[p].parent
    goto upward
}
if inputnode[p].left ≡ q {
    q, p, d = p, inputnode[p].right, 1
    goto downward
}
if inputnode[p].right ≡ q {
    q, p, d = p, inputnode[p].left, −1
    goto downward
}
confusion("supposed parent node not apparent")
```

이 코드는 21 절에서 쓰인다.

24. 내려갈 때는 언제나 가운데 이음줄을 탄다. 빈자리(= 0)에 이르면 거기가 짝의 자리이므로, 건너뛴 쪽에 맞추어 새 싹을 꽂는다.

⟨ 가운데를 따라 내려가 짝을 놓는다 24 ⟩ ≡

```
downwardmid:
q, p, d = p, inputnode[p].middle, 0
downward:
if p < 0 {
    abort0("짝이 엉켰다", 25)
}
if p > 0 {
    goto downwardmid
}
if d > 0 {
    inputnode[q].right = -buds
} else if d < 0 {
    inputnode[q].left = -buds
} else {
    inputnode[q].middle = -buds
}
inputbud[buds].parent = q
```

이 코드는 21 절에서 쓰인다.

25. 싹을 채우고 계급을 매기는 일은 *fillbuds* 라는 곧이곧대로의 되도는 함수가 한다. 삼진 나무를 앞선 차례(preorder)로 훑는다.

⟨ 함수들 17 ⟩ +≡

```
func fillbuds(p, r int) {
    if r < 0 {
        fmt.Fprintf(os.Stderr, "제대로 비스듬하지 않다: rank(%c)=-1!\n", p)
        os.Exit(30)
    }
    inputnode[p].rank = r
    ⟨ 세 자식을 차례로 훑으며 빈자리에 싹을 넣는다 26 ⟩
}
```

26. 세 자식이 하는 일이 꼴만 같고 계급이 저마다 다르다. 자식이 있으면 그리로 내려가고, 없으면 그 자리에 싹을 넣는다. 이미 싹이 있으면(< 0) 계급만 적어 둔다.

⟨ 세 자식을 차례로 훑으며 빈자리에 싹을 넣는다 26 ⟩ ≡

```
if inputnode[p].left > 0 {
    fillbuds(inputnode[p].left, r − 1)
} else {
    if inputnode[p].left ≡ 0 {
        inputnode[p].left = −buds − 1
        setmate(p)
    }
    inputbud[−inputnode[p].left].rank = r − 1
}
if inputnode[p].middle > 0 {
    fillbuds(inputnode[p].middle, r)
} else {
    if inputnode[p].middle ≡ 0 {
        inputnode[p].middle = −buds − 1
        setmate(p)
    }
    inputbud[−inputnode[p].middle].rank = r
}
if inputnode[p].right > 0 {
    fillbuds(inputnode[p].right, r + 1)
} else {
    if inputnode[p].right ≡ 0 {
        inputnode[p].right = −buds − 1
        setmate(p)
    }
    inputbud[−inputnode[p].right].rank = r + 1
}
```

이 코드는 25 절에서 쓰인다.

27. 빠진 밭을 모두 채워 비스듬한 삼진 나무를 마련했다. 그런데 들어온 것이 이를테면 '`A--- B-B-`'라면, 고리가 생겨서 마련한 나무에 주어진 마디가 다 들어 있지 않다. 그러니 찾은 싹이 $2n + 2$ 개인지 반드시 확인해야 한다.

⟨ 나무를 온전히 채웠는지 살핀다 27 ⟩ ≡

```
if buds ≠ n + n + 2 {
    abort0("들어온␣것에␣고리가␣있다", 66)
}
```

이 코드는 20 절에서 쓰인다.

28. 상태표. 나무를 메모리에 들여놓았으니 이제 앨리스의 걸음을 흉내 낼 수 있다. 이 프로그램은 꼭 필요한 것보다 많은 것을 모아 둔다. 여분의 자료가 구조의 성질을 알아내는 데 도움이 될까 해서라고 커누스가 적어 두었다.

⟨ 자료형 14 ⟩ +≡

```
type step struct {
    rank int    // 들어설 때의 계급
    first int    // 지나는 싹, 또는 호의 첫 마디
    second int    // (둘째 경우에) 호의 끝 마디
    match int    // (둘째 경우에) 맞붙는 싹
}
```

29. ⟨ 전역 변수 15 ⟩ +≡

```
var (
    chart [4 * maxcodes]step    // 길이 4n 짜리 상태표
    steps int    // 지금 chart 에 든 항목 수
    stack [256]int    // 아직 맞붙지 못한 싹들
    stacked int    // 그런 싹의 수
)
```

30. ⟨ T 의 켤레 셋을 찾아 내놓는다 30 ⟩ ≡

⟨ 상태표를 만든다 35 ⟩
⟨ 싹까지 모두 드러낸 나무를 내놓는다 38 ⟩
⟨ 상태표에서 켤레들을 내놓는다 37 ⟩

이 코드는 11 절에서 쓰인다.

31. 상태표는 *fillbuds* 와 비슷한 되도는 함수 *createsteps* 가 만든다.

⟨ 함수들 17 ⟩ +≡

```
func createsteps(p int) {
    q := inputnode[p].left
    if q > 0 {
        branch(p, q)
    } else {
        budstep(−q)
    }
    q = inputnode[p].middle
    if q > 0 {
        branch(p, q)
    } else {
        budstep(−q)
    }
    q = inputnode[p].right
    if q > 0 {
        branch(p, q)
    } else {
        budstep(−q)
    }
}
```

32. 쌓인 싹의 수 *stacked* 와 지금 계급의 차이가 *offset* 이다.

⟨ 상수 13 ⟩ +≡

```
const offset = 2
```

33. ⟨ 함수들 17 ⟩ +≡

```
func budstep(b int) {   // 상태표에 싹이 하나 더해진다
    chart[steps].first, chart[steps].rank = b, stacked − offset
    if chart[steps].rank ≠ inputbud[b].rank {
        confusion("rank␣offense␣b")
    }
    inputbud[b].stepno = steps
    steps++
    stack[stacked] = b
    stacked++
}
```

34. ⟨ 함수들 17 ⟩ +≡

```
func branch(p, q int) {   // 상태표가 호에서 그 쌍대 호로 건너간다
    chart[steps].first, chart[steps].second = p, q
    chart[steps].rank = stacked − offset
    if chart[steps].rank ≠ inputnode[q].rank {
        confusion("rank offense q")
    }
    stacked −−
    chart[steps].match = stack[stacked]
    steps ++
    createsteps(q)
    chart[steps].first, chart[steps].second = q, p
    chart[steps].rank = stacked − offset
    if chart[steps].rank ≠ inputnode[q].rank + 2 {
        confusion("rank offense p")
    }
    stacked −−
    chart[steps].match = stack[stacked]
    steps ++
}
```

35. ⟨ 상태표를 만든다 35 ⟩ ≡

```
chart[0].rank, chart[0].first, steps = −2, 1, 1
stack[0], stacked = 1, offset − 1
createsteps(inputbud[1].parent)
if stacked ≠ 2 + offset {
    confusion("mismatched")
}
if steps ≠ 4 ∗ n {
    confusion("total steps")
}
```

이 코드는 30 절에서 쓰인다.

36. 거꾸로, 상태표가 주어지면 맞붙지 못한 싹 다음에서 시작하는 나무를 내놓는 되도는 함수를 간단히 지을 수 있다. 재미있게도, 마디를 만나는 것은 앞선 차례인데 알리는 것은 뒤선 차례(postorder)다.

```
⟨ 함수들 17 ⟩ +≡
  func printfam(p int) {
      var l, m, r int
      if steps ≡ 4 * n {
          steps = 0
      }
      q := chart[steps].second
      steps++
      if q ≡ 0 {
          l = '-'
      } else {
          l = q
          printfam(q)
          steps++
      }
      if steps ≡ 4 * n {
          steps = 0
      }
      q = chart[steps].second
      steps++
      if q ≡ 0 {
          m = '-'
      } else {
          m = q
          printfam(q)
          steps++
      }
      if steps ≡ 4 * n {
          steps = 0
      }
      q = chart[steps].second
      steps++
      if q ≡ 0 {
          r = '-'
      } else {
          r = q
          printfam(q)
          steps++
      }
      fmt.Printf("␣%c%c%c%c", p, l, m, r)
  }
```

37. 여기 알고리즘은 아주 귀엽다. 풀어내는 재미는 독자에게 남겨 둔다고 커누스가 적었으므로, 나도 그러기로 한다.

⟨ 상태표에서 켤레들을 내놓는다 37 ⟩ ≡

```
chart[4 * n].second, chart[4 * n].first = sentinel, inputbud[1].parent
for j = 1; j < 4; j++ {
    fmt.Print(strings.Repeat("+", j))
    fmt.Print(":")
    steps = inputbud[stack[j + offset − 2]].stepno + 1
    for k = steps; chart[k].second ≡ 0; k++ {
    }
    printfam(chart[k].first)
    fmt.Println()
    if chart[steps].first ≠ stack[j + offset − 2] {
        confusion("bad␣end␣of␣cycle")
    }
}
```

이 코드는 30 절에서 쓰인다.

38. ⟨ 싹까지 모두 드러낸 나무를 내놓는다 38 ⟩ ≡

```
printTree(inputbud[1].parent)
fmt.Println()
```

이 코드는 30 절에서 쓰인다.

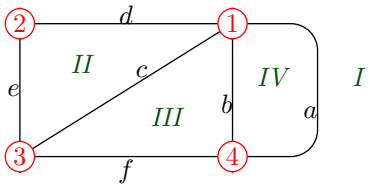
39. 마디 p 의 부분나무를 앞선 차례로 알린다. 계급만큼 들여쓰므로 나무의 모양이 눈에 들어온다.

⟨ 함수들 17 ⟩ +≡

```
func printTree(p int) {
    fmt.Print(strings.Repeat(".", inputnode[p].rank + 8))
    fmt.Printf("␣%c:", p)
    if inputnode[p].left < 0 {
        fmt.Printf("%3d", -inputnode[p].left)
    } else {
        fmt.Printf("␣␣%c", inputnode[p].left)
    }
    if inputnode[p].middle < 0 {
        fmt.Printf("%3d", -inputnode[p].middle)
    } else {
        fmt.Printf("␣%c␣", inputnode[p].middle)
    }
    if inputnode[p].right < 0 {
        fmt.Printf("%3d\n", -inputnode[p].right)
    } else {
        fmt.Printf("␣%c\n", inputnode[p].right)
    }
    if inputnode[p].left > 0 {
        printTree(inputnode[p].left)
    }
    if inputnode[p].middle > 0 {
        printTree(inputnode[p].middle)
    }
    if inputnode[p].right > 0 {
        printTree(inputnode[p].right)
    }
}
```

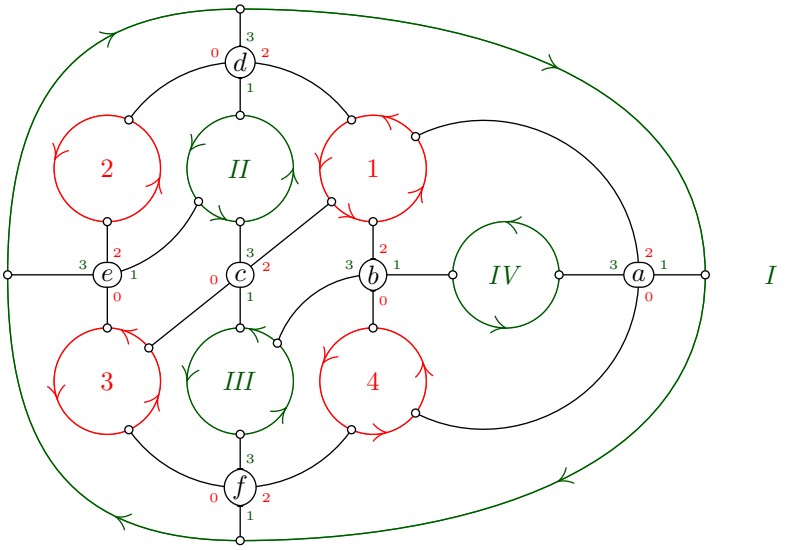
40. 평면 지도를 담는 quad-edge 자료 구조. 나무를 떠나 평면에 그린 더 복잡한 그래프로 옮겨 가자. 여기서는 Leo Guibas와 Jorge Stolfi가 내놓은 아름다운 자료 구조를 쓴다(*ACM Transactions on Graphics* **4** (1985), 74–123).

그들의 "quad-edge 구조"를 이해하는 가장 좋은 길은 작은 보기를 보는 것이다. 꼭짓점이 $\{1,2,3,4\}$이고 변이 $\{a,b,c,d,e,f\}$이고 면이 $\{\mathrm{I},\mathrm{II},\mathrm{III},\mathrm{IV}\}$인 평면 그래프를 흔히 그리는 방법은, 변을 선으로 그어 꼭짓점을 잇고 둘러싸인 자리에 면 이름을 적는 것이다.



(∗)

그런데 컴퓨터 안에서는 이 그림의 위상을 나타내는 가장 좋은 길이 좀 더 공들인 구조를 짓는 것이다. 그것은 그래프 (∗)에 주석을 달아 더 풍부한 그래프에 앉힌 것으로 볼 수 있다.

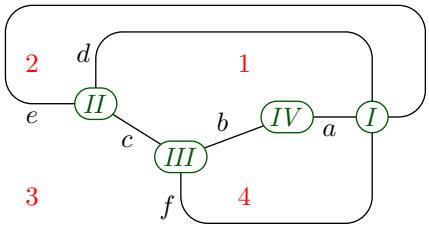


(∗∗)

꼭짓점(빨강)과 면(초록)이 방향 있는 고리로 바뀌었다. 고리는 모두 반시계 방향으로 도는데, 가장 바깥 고리만 시계 방향이다. (구 위의 적도에 그려 놓고 남극에서 바라보면 그 고리도 반대로 돌 것이다. 나머지는 북극에서 보고 있는 셈이다.)

41. (∗∗)의 방향 있는 고리에는 **꼭지**(pip)라 부를 작은 이음매가 달려 있다. 차수가 d인 꼭짓점 v의 고리에는 꼭지가 d개 있어서 v에 닿은 변을 모두 반시계 차례로 가리킨다. 면의 고리도 마찬가지로 그 면을 둘러싼 변들을 반시계로 돌며 가리킨다.

(∗∗) 같은 표현의 좋은 점 하나는 **쌍대** 그래프도 함께 나타낸다는 것이다. 쌍대에서는 꼭짓점이 면이 되고 면이 꼭짓점이 되며 변이 90° "돈다". 보기를 들어 (∗)의 쌍대는 이 평면 그래프다.



. (∗∗∗)

42. (∗)의 변 $\{a, b, c, d, e, f\}$이 저마다 (∗∗)에서는 꼭짓점으로 나타남을 눈여겨보라. 그런 꼭짓점은 모두 차수가 4이고, 시계 방향으로 0, 1, 2, 3이라 번호 매긴 선을 거쳐 꼭지 넷에 이어진다. 0번과 2번 선의 꼭지는 언제나 꼭짓점 고리에 속하고, 1번과 3번 선의 꼭지는 언제나 면 고리에 속한다. 변마다 $(0, 1, 2, 3)$을 $(2, 3, 0, 1)$로 바꾸어도 이 그림의 뜻은 달라지지 않는다. 번호 자체는 중요하지 않다. 하지만 고리 차례는 매우 중요하고, 홀짝도 그렇다.

43. 짚어 둘 것이 있다. 평면 그래프 둘은 구 위에 그렸을 때 위상이 같으면 본질적으로 같다고 본다. 구면을 주어진 변들이 나누는 모양이 같아야 하고, 한쪽을 다른 쪽으로 매끄럽게 속임수 없이 옮길 수 있어야 한다는 뜻이다. 특히 (∗)는 바깥 면으로 어느 면을 고르느냐에 따라 세 가지로 다시 그릴 수 있다.

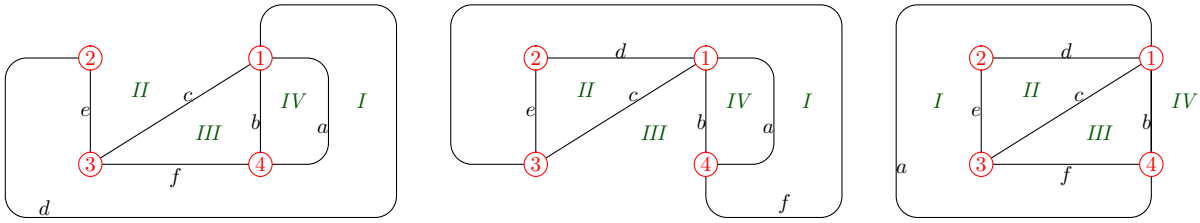


그림 5: 같은 평면 지도를 바깥 면만 바꾸어 다시 그린 것. 왼쪽부터 II, III, IV가 바깥이다.

이들은 저마다 (∗∗)의 꼭짓점과 변과 면과 꼭지에 꼭 들어맞는다. 그러니 (∗∗)의 변종 셋인 셈이다. 다만 좌우를 뒤집으면 다른 그래프가 된다. (∗)에 대칭이 없기 때문이다.

44. 변이 m개라 하자. 그림 (∗∗)는 꼭지 $4m$개의 **치환**으로도 볼 수 있고, 고리 꼴로 적으면 이렇다.

$$\alpha = (a_2d_2c_2b_2)(d_0e_2)(c_0e_0f_0)(a_0b_0f_2)(a_1f_1e_3d_3)(c_3d_1e_1)(b_3c_1f_3)(a_3b_1).$$

이 고리들이 꼭짓점 (1), (2), (3), (4)와 면 (I), (II), (III), (IV)에 해당한다. 보기를 들어 '$(a_0b_0f_2)$'는 (∗∗)에서 꼭짓점 4의 고리 $a_0 \longrightarrow b_0 \longrightarrow f_2 \longrightarrow a_0$을 나타낸다.

45. Guibas와 Stolfi는 평면 지도에서 이렇게 얻은 치환 α가 아주 중요한 성질을 지님을 알아챘다. **되돌아가기 공리**다. α가 $u_{i+1} \mapsto v_j$로 보낸다면—여기서 u와 v는 나타내려는 평면 그래프의 변이고 아래 첨자는 4를 법으로 다룬다—α는 $v_{j+1} \mapsto u_i$로도 보낸다. 보기를 들어 α에서 $a_2 \mapsto d_2$이고 $d_3 \mapsto a_1$이다. a_2와 d_2는 꼭짓점 꼭지인데 d_3과 a_1은 면 꼭지임을 눈여겨보라.

46. 되돌아가기 공리는 특별한 "돌리기" 치환

$$\rho = (a_0a_1a_2a_3)(b_0b_1b_2b_3)(c_0c_1c_2c_3)(d_0d_1d_2d_3)(e_0e_1e_2e_3)(f_0f_1f_2f_3)$$

를 써서 치환의 말로 옮길 수 있다. 곧 $\alpha\rho\alpha\rho$가 항등 치환이라는 말과 같다. 어느 꼭지에서든 α를 쓰고 ρ를 쓴 다음, α와 ρ를 한 번 더 쓰면 처음 자리로 돌아온다. quad-edge 구조가 빠른 밑바탕이 여기 있다. 어떤 꼭짓점 둘레든 어떤 면 둘레든 시계 방향으로도 반시계 방향으로도 쉽게 옮겨 다닐 수 있고, 앞선 것을 찾으려고 고리를 통째로 훑을 까닭이 없다.

47. 보기를 이어 가면

$$\alpha\rho = (a_0b_1)(a_1f_2)(a_2d_3)(a_3b_2)(b_0f_3)(b_3c_2)(c_0e_1)(c_1f_0)(c_3d_2)(d_0e_3)(d_1e_2)(e_0f_1)$$

이다. 일반적으로 $\alpha\rho$는 언제나 차수 2의 치환이고, 꼭짓점 꼭지를 모두 어떤 면 꼭지로 보낸다. 그러니 $\alpha\rho$는 온통 두 고리로만 이루어지고, 꼭짓점 꼭지 $2m$개와 면 꼭지 $2m$개 사이의 짝짓기다.

마찬가지로 $\rho\alpha$도 언제나 두 고리로만 이루어지는데, 우리 경우에는

$$\rho\alpha = (a_0f_1)(a_1d_2)(a_2b_1)(a_3b_0)(b_3f_2)(b_2c_1)(c_3e_0)(c_0f_3)(c_2d_1)(d_3e_2)(d_0e_1)(e_3f_0)$$

이다. 되돌아가기 공리 때문에, $\rho\alpha$에 (u_iv_j)가 있는 것과 $\alpha\rho$에 $(u_{i+1}v_{j+1})$이 있는 것이 같은 말이다.

$(**)$에서 고리 바깥에 남은 자리는 모두 네 변을 지닌다. 보기를 들어 오른쪽 위쯤에 모퉁이 꼭지가 반시계로 (d_3, d_2, a_2, a_1)인 자리가 있다. 되돌아가기 공리가 이 사실을 설명해 준다. 나아가 $\{d_3, a_2\}$와 $\{d_2, a_1\}$처럼 마주 보는 모퉁이의 꼭지가 바로 $\alpha\rho$와 $\rho\alpha$가 짝지어 주는 것들임도 말해 준다.

48. 그러니 변이 m개인 평면 그래프의 quad-edge 자료 구조는 본질적으로 포인터 $4m$개다. 그것이 한 꼭지에서 다른 꼭지로 어떻게 옮겨 가는지 알려 준다. 이 포인터들은 꼭지의 치환을 이루는데, 그 치환은 꼭짓점 꼭지를 꼭짓점 꼭지로, 면 꼭지를 면 꼭지로 보내고 되돌아가기 공리도 채운다.

49. 꼭지 하나는 변 이름의 ASCII 코드를 넷 곱한 것에 아래 첨자를 더해 나타내면 편하다. 보기를 들어 a_3은 `'a'` $\ll 2+3$이고, `'a'`가 97이므로 391이다.

메모리에는 α와 그 역 α^-를 함께 둔다. 둘 다 쓸모가 있기 때문이다. (꼭 둘 다 둘 까닭은 없다. 되돌아가기 공리 $\alpha^- = \rho\alpha\rho$가 언제나 성립하니까.)

돌리기 ρ는 아래 두 비트만 4를 법으로 올리는 일이다. 커누스는 가지치기를 피하려고 배타적 논리합으로 쓴 비트 재주를 썼는데, 여기서는 뜻이 곧바로 드러나도록 적었다. 하는 일은 같다.

⟨ 함수들 17 ⟩ +≡

```
func pip(u, i int) int { return u ≪ 2 + i }
func pipEdge(p int) int { return p ≫ 2 }
func pipSub(p int) int { return p & 3 }
func rot(p int) int { return p ⊘ 3 | (p + 1) & 3 }   // ρ
func irot(p int) int { return p ⊘ 3 | (p + 3) & 3 }   // ρ⁻
```

50. ⟨ 전역 변수 15 ⟩ +≡

```
var (
    alpha [4 * 256]int    // 지금 평면 지도를 나타내는 꼭지의 치환
    alphainv [4 * 256]int    // 그 역
    verts int    // 지금 꼭짓점의 수
)
```

51. 꼭지를 뒤섞는 것은 특별한 **뿌리 변** 하나와, 들어온 비스듬한 삼진 나무의 마디에 해당하는 변들뿐이다. 그런 마디는 왼쪽 자식이 있다는 것으로 가려낼 수 있다. (가운데 자식과 오른쪽 자식도 있지만 그것이 무엇인지는 아무래도 좋고 있다는 것만 중요하다.) 뿌리 변의 이름은 *다.

⟨ 초기 치환을 만든다 51 ⟩ ≡

```
verts = 0
for k = '*'; k ≤ '~'; k++ {
    if k ≠ '*' ∧ inputnode[k].left ≡ 0 {
        continue
    }
    alpha[pip(k,0)], alphainv[pip(k,0)] = pip(k,0), pip(k,0)
    alpha[pip(k,1)], alphainv[pip(k,1)] = pip(k,3), pip(k,3)
    alpha[pip(k,2)], alphainv[pip(k,2)] = pip(k,2), pip(k,2)
    alpha[pip(k,3)], alphainv[pip(k,3)] = pip(k,1), pip(k,1)
    verts += 2
}
if verts ≠ 2 * (n + 1) {
    confusion("initial␣vertex␣count")
}
```

이 코드는 72, 85 절에서 쓰인다.

52. 함수 *splice* 는 서로 바꿔야 할 꼭짓점 꼭지 둘의 자리를 받는다. 그리고 위에서 말한 이어붙이기 규칙으로 그에 맞는 면 꼭지 둘을 알아낸다.

이어붙이기가 "적법한지"는—평면성을 지키는지는—따지지 않는다. 꼭짓점의 수를 줄이는 데에만 *splice* 를 쓸 것이기 때문이다. 적법하지 않게 쓰면 마지막에 세는 면의 수가 오일러의 잣대와 맞지 않게 되어 드러난다.

(딱 한 군데 예외가 있다. 아래에서 같은 꼭짓점 고리에 이웃해 있는 꼭지 둘을 갈라 놓는 자리가 있는데, 그때는 *verts* 를 2 늘려 셈을 맞춘다.)

```
⟨ 함수들 17 ⟩ +≡
  func splice(p, q int) {
      if (p & 1) + (q & 1) ≠ 0 {
          confusion("attempt to splice face pips")
      }
      r, s := alphainv[p], alphainv[q]
      alphainv[p], alphainv[q] = s, r
      alpha[s], alpha[r] = p, q
      p, q = alpha[rot(p)], alpha[rot(q)]   // 이제 알맞은 면들을 맞바꾼다
      r, s = alphainv[p], alphainv[q]
      alphainv[p], alphainv[q] = s, r
      alpha[s], alpha[r] = p, q
      verts --
  }
```

53. α의 고리를 찍어서 평면 그래프에 대한 쓸 만한 정보를 모두 보여 주는 귀여운 함수다. 먼저 꼭짓점 고리의 꼭지들을 한 줄에 하나씩, 이어서 면 고리의 꼭지들을 한 줄에 하나씩 찍는다.

꼭짓점과 면의 수도 세어 두었다가 오일러의 공식으로 성분의 수를 알린다(평면이라 가정하고).

```
⟨ 함수들 17 ⟩ +≡
  func printAlpha() {
      var c, f, v int
      ⟨ 꼭짓점 고리를 찍고 센다 54 ⟩
      if v ≠ verts {
          confusion("vertex count")
      }
      ⟨ 면 고리를 찍고 센다 55 ⟩
      c = (v − (n + 1) + f) ≫ 1
      fmt.Printf("(모두 꼭짓점 %d개, 변 %d개, 면 %d개, 성분 %d개다.)\n",
          v, n + 1, f, c)
  }
```

54. 생각은 이렇다. 아직 찍지 않은 고리의 우두머리(그런 p 가운데 가장 작은 것)를 찾아 그 고리를 찍는 일을, 짝수 번호 꼭지의 고리를 다 찾을 때까지 되풀이한다. 찍은 자리는 알고리즘 1.3.3I처럼 잠깐 음수로 뒤집어 표시해 둔다.

⟨ 꼭짓점 고리를 찍고 센다 54 ⟩ ≡

```
fmt.Println("꼭짓점:")
v = 0
p := pip('*', 0)
t := 2 * (n + 1)
for {
    for; alpha[p] ≤ 0 ∧ t ≠ 0; p += 2 {
        if alpha[p] < 0 {
            alpha[p] = -alpha[p]
            t--
        }
    }
    if t ≡ 0 {
        break   // 아직 못 본 꼭지가 t 개 남아 있다
    }
    for q, r := p, alpha[p]; r > 0; q, r = r, alpha[r] {
        fmt.Printf("␣%c%d", pipEdge(r), pipSub(r))
        alpha[q] = -r
    }
    fmt.Println()
    v++
}
```

이 코드는 53 절에서 쓰인다.

55. 홀수 번호 꼭지도 똑같은 생각으로 다룬다.

⟨ 면 고리를 찍고 센다 55 ⟩ ≡

```
fmt.Println("면:")
f = 0
p = pip('*', 1)
t = 2 * (n + 1)
for {
    for; alpha[p] ≤ 0 ∧ t ≠ 0; p += 2 {
        if alpha[p] < 0 {
            alpha[p] = -alpha[p]
            t--
        }
    }
    if t ≡ 0 {
        break
    }
    for q, r := p, alpha[p]; r > 0; q, r = r, alpha[r] {
        fmt.Printf("␣%c%d", pipEdge(r), pipSub(r))
        alpha[q] = -r
    }
    fmt.Println()
    f++
}
```

이 코드는 53 절에서 쓰인다.

56. 평면 그래프의 벽돌. 이어진 다중그래프는 모두, 이른바 **블록**(biconnected component, 또는 나눌 수 없는 그래프)들이 **관절점**(articulation point, 또는 자름 꼭짓점)으로 붙어 나무처럼 엮인 것이다. 꼭짓점이 둘보다 적은 시시한 경우는 뺀다. 곧 빈 그래프와 꼭짓점 하나짜리 K_1과 제 고리 하나뿐인 다중그래프는 다루지 않는다.

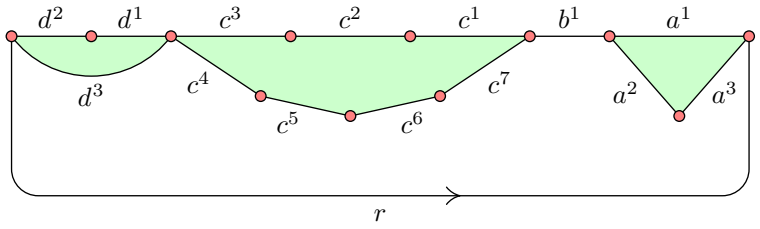
시시하지 않은 이중연결 평면 그래프에 **뿌리를 매긴다**는 것은 변 하나에 화살표를 얹어 그것을 u에서 v로 가는 방향 있는 호로 삼는 일이다. 그 변을 뿌리 변이라 하고 꼭짓점 u를 뿌리라 한다. 그리고 뿌리 변이 바깥 면을 반시계로 도는 길 위에 오도록 그래프를 그린다. (달리 말해 u에서 v로 갈 때 바깥 면이 오른쪽에 있다.)

뿌리를 매긴 시시하지 않은 이중연결 평면 지도—앞으로 RNBPM이라 적겠다—란 그런 그래프들의 동치류다. 구 위에 그렸을 때 위상이 같은 둘을 같다고 본다. 그러니 RNBPM은 저마다 α 치환으로 나타낼 수 있다. 다만 변 이름을 바꾸는 것과, 고른 변 몇몇의 아래 첨자에 2를 (4를 법으로) 더하는 것은 같은 것으로 본다.

57. 뿌리 변을 r라 하고 바깥 고리의 나머지 변을 $e^1, e^2, \ldots, e^p$라 하자. 뿌리 꼭짓점 고리가 꼭지 r_0을 품도록 정할 것이고, 따라서 바깥 면 고리는 꼭지 r_3을 품는다. 또 α가 $r_0 \mapsto e_2^1$, $e_0^1 \mapsto e_2^2$, $\ldots$, $e_0^p \mapsto r_2$로 보내도록 꼭지 번호를 정한다. 그러면 바깥 면 고리는 $(e_3^p \ldots e_3^2 e_3^1 r_3)$이다. 다만 $p = 0$이면 그 고리는 물론 $(r_1 r_3)$이다.

58. 가장 간단한 RNBPM은 뿌리 변 하나뿐인 것이다. 그렇지 않으면 어떤 RNBPM이든 간단한 방법으로 되돌아가며 지을 수 있다. 뿌리 변을 떼어 내면 블록이 $m \geq 1$개인 그래프가 남고(따라서 관절점이 $m-1$개다), 그 블록마다 뿌리 변을 정해 주면 저마다 RNBPM이 된다. 뿌리로는 전체 그래프의 바깥 면을 반시계로 돌 때 처음 만나는 변을 고른다. 그리고 전체 그래프에서 바깥이 **아닌** 첫 변도 적어 두어 그 블록에 "두 겹 뿌리"를 매긴다. 그러면 원래 RNBPM을 두 겹 뿌리 블록들에서 쉽게 되살릴 수 있다.

59. 또 보기가 간절하다. 앞의 그래프 $(*)$도 변 하나를 뿌리로 삼으면 RNBPM이 되지만, 그것은 너무 간단해서 일반적인 형편이 드러나지 않는다. 그래서 조금 더 복잡한 것을 보자.



(†)

그림 6: RNBPM 하나를 뿌리 변 r를 떼어 블록 넷으로 나눈 모습. 음영을 넣은 블록은 바깥에서 보이지 않는 복잡한 속을 품고 있을 수 있다.

여기서 $m = 4$이고 바깥 면에는 다른 변이 $p = 7$개 있다. 그러니 그 고리는 $(r_3 d_3^2 d_3^1 c_3^3 c_3^2 c_3^1 b_3^1 a_3^1)$이다. 또 r에 닿은 안쪽 면은 $(r_1 a_3^3 a_3^2 b_1^1 c_3^7 c_3^6 c_3^5 c_3^4 d_3^3)$이다. 변 r를 떼어 내면 관절점이 셋 솟아나 남은 그래프를 블록 넷으로 나눈다. 바깥 변이 $\{a, b, c, d\}$인 것들이다. 블록 b는 다리 하나뿐이지만 나머지는 더 작은 것들로 지어졌다.

60. 이 네 블록은 뿌리 변 꼭지가 각각 a_3^1, b_3^1, c_3^1, d_3^1인 RNBPM으로 볼 수 있다. 그리고 뿌리가 아닌 바깥 꼭짓점 꼭지 a_2^2, b_0^1, c_2^4, d_2^3를 정해 주므로 두 겹 뿌리이기도 하다. 그것이 블록들을 어떻게 이어 붙일지 말해 준다. $\alpha, \beta, \gamma, \delta$가 그 블록들의 치환이고 $\omega = (r_0)(r_2)(r_1 r_3)$이 변 r의 치환이라면, 전체 RNBPM의 치환은 $\alpha\beta\gamma\delta\omega\,\sigma_1\sigma_2\sigma_3\sigma_4\sigma_5$이고

$$\sigma_1 = (d_2^3 r_2)(d_3^2 r_1), \quad \sigma_2 = (c_2^4 d_2^1)(c_3^3 d_3^3), \quad \sigma_3 = (b_0^1 c_2^1)(b_3^1 c_3^7), \quad \sigma_4 = (a_2^2 b_2^1)(a_3^1 b_1^1), \quad \sigma_5 = (a_2^1 r_0)(r_3 a_3^3)$$

가 알맞은 이어붙이기다.

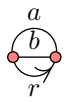
61. 손쉽게 견주어 볼 수 있도록, 가장 작은 RNBPM들과 그 표준 치환을 적어 둔다.



$(r_0)(r_2)(r_3r_1)$



$(r_0a_2)(a_0r_2)(r_3a_3)(a_1r_1)$



$(r_0a_2b_0)(a_0r_2b_2)(r_3a_3)(a_1b_1)(b_3r_1)$



$(r_0a_2)(a_0b_2)(b_0r_2)(r_3b_3a_3)(a_1b_1r_1)$



$(r_0a_2c_2b_0)(a_0r_2b_2c_0)(r_3a_3)(a_1c_3)(c_1b_1)(b_3r_1)$



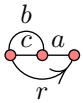
$(r_0a_2c_0)(a_0b_2)(b_0r_2c_2)(r_3b_3a_3)(a_1b_1c_1)(c_3r_1)$



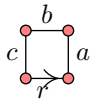
$(r_0a_2c_0)(a_0r_2b_2)(b_0c_2)(r_3a_3)(a_1b_1c_1)(c_3b_3r_1)$



$(r_0a_2c_0)(a_0b_2c_2)(b_0r_2)(r_3b_3a_3)(a_1c_1)(c_3b_1r_1)$



$(r_0a_2)(a_0b_2c_0)(b_0r_2c_2)(r_3b_3a_3)(b_1c_1)(a_1c_3r_1)$



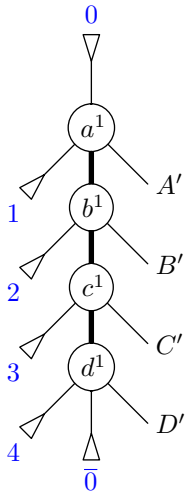
$(r_0a_2)(a_0b_2)(b_0c_2)(c_0r_2)(r_3c_3b_3a_3)(r_1a_1b_1c_1)$

62. Jacquard와 Schaeffer의 평면 지도. 이제 비스듬한 삼진 나무라는 본래 이야기로 돌아온다.

맨 처음에 Del Lungo 들이 비스듬한 삼진 나무와 RNBPM 사이의 흥미로운 대응을 찾아냈다고 말했다. 그들은 비스듬한 삼진 나무라는 생각을 먼저 지어내고, 마디가 n개인 그런 나무의 수가 뿌리 아닌 변이 n개인 RNBPM의 수와 꼭 같으리라고 짐작한 다음에 그 대응을 찾았다.

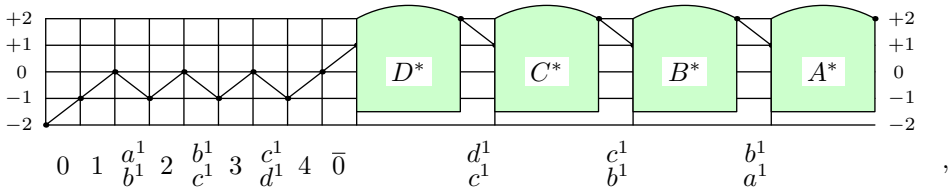
Benjamin Jacquard와 Gilles Schaeffer는 그 짐작에 답하면서, Del Lungo 들이 거의 같은 때에 찾아낸 것과는 사뭇 다른 기발한 대응을 내놓았다(*Journal of Combinatorial Theory* **A83** (1998), 1–20). 커누스는 두 대응이 어떻게든 이어져 있는지 궁금해서 이 프로그램에 둘 다 넣었다.

63. 그들의 구성에 따르면 (†) 같은 RNBPM은 이런 꼴의 비스듬한 삼진 나무로 나타난다.



,

여기서 A', B', C', D'는 변 r를 떼었을 때 나오는 블록 $m = 4$개의 두 겹 뿌리 RNBPM들이다. 그러니 그들의 표현에 대응하는 상태표는 이런 꼴이 된다.



,

여기서 A^*, B^*, C^*, D^*는 부분나무 A', B', C', D'를 어떤 식으로든 나타낸 것이다.

이 보기에서 B'는 비어 있다. (†)에서 성분 b에 변이 하나뿐이기 때문이다. 그래서 B^*는 싹 $\overline{2}$에 대한 "+1 걸음" 하나일 뿐이다. 그러나 A', C', D'는 비어 있지 않다(사실 아주 복잡할 수도 있다).

64. 구성을 마치려면 두 겹 뿌리 RNBPM을 어떻게 나타낼지 밝혀야 한다. 보기를 들어 이 프로그램 맨 앞에 나온 비스듬한 삼진 나무 T를 보자. T에 대응하는 RNBPM은 더 큰 RNBPM을 짓는 데 세 가지로 쓰일 수 있다. T에 계급 0인 마디 `A`, `B`, `E`가 셋 있기 때문이다.

앞에서 다룬 "싹과 상태표" 방법이 둘째 뿌리를 담아내는 좋은 길을 준다. 계급이 -1인 싹에서 시작하는 고리 변종 하나를 쓰는 것이다(싹 1, 2, 4번). 그 나무들에는 계급 -1인 마디가 각각 2개, 1개, 0개 있고 계급 -2인 마디는 없다. 그래서 계급이 0인 마디의 오른쪽 부분나무 T'로 안심하고 쓸 수 있다.

65. 보기를 들어 이 경우 T^*가 될 수 있는 셋의 상태표는 각각 이렇다.

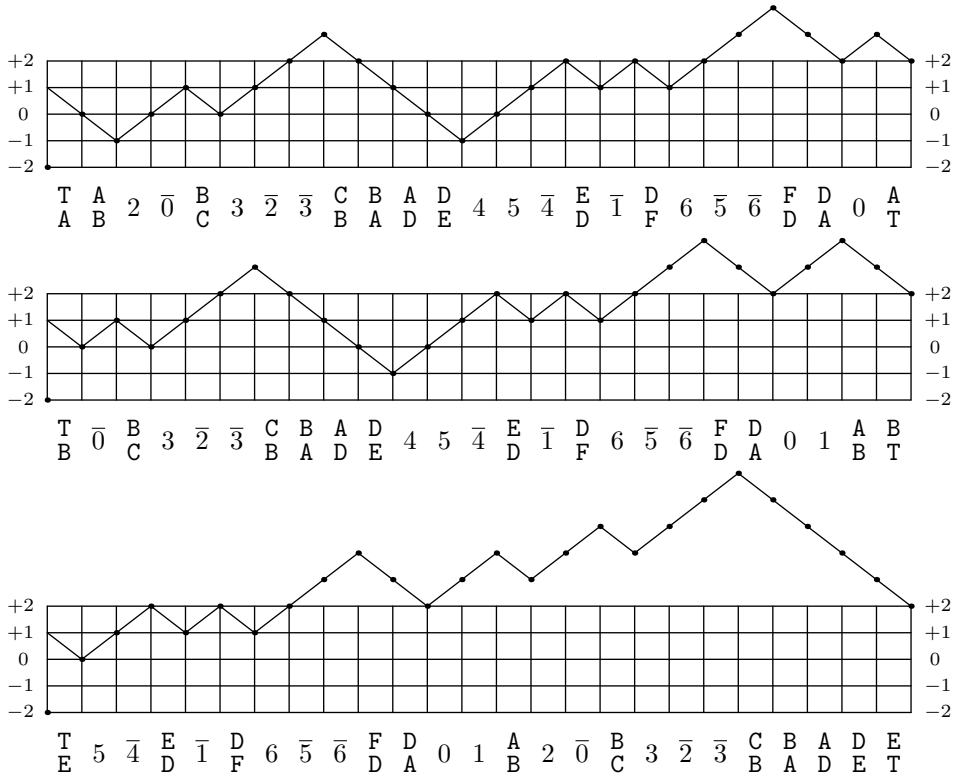


그림 7: 계급 -1인 싹 1, 2, 4번에서 시작하는 세 변종의 상태표.

만드는 한 방법은 그 싹에서 시작해 같은 싹이 다시 나올 때까지 고리처럼 상태표를 이어 만드는 것이다. 그러고는 그 싹이 나온 두 자리를 지우고, 가상의 부모 마디 `T`에서 새 부분나무 뿌리로 갔다 돌아오는 맞물린 호로 바꿔 놓는다. (따라서 마디가 n개인 부분나무 T'의 상태표 T^*는 길이가 $4n+1$이다. $n=0$일 때도 그렇다.)

거꾸로 그 옮긴 변종 T^*에서 T를 되찾는 것도 쉽다. 먼저 전체를 감싼 맞물린 호를 지우고, 지웠던 싹을 되돌려 놓는다. 그다음 계급 -2인 싹이 처음 생길 때까지 고리를 되감는다. (그 싹은 가장 오른쪽에 있는 계급 $+2$인 싹에서 복제된다.)

이 구성에서 비스듬한 삼진 나무의 홀수 계급 마디의 수는 대응하는 RNBPM의 면의 수에서 2를 뺀 것과 같음을, 그리고 짝수 계급 마디의 수는 뿌리 아닌 꼭짓점의 수와 같음을 귀납법으로 밝힐 수 있다.

66. 우리의 주된 목표는 주어진 비스듬한 삼진 나무에서 대응하는 RNBPM을 지어내는 것인데, 그 평면 지도의 quad-edge 치환을 셈해서 한다. 나무는 상태표 꼴로 주어진다. 메모리는 아끼지 않기로 하고, 되돌기 중에 생기는 여러 상태표를 쌓아 둔다.

마디가 n개인 나무는 뿌리 변 말고도 변이 n개인 RNBPM을 낸다.

⟨ 전역 변수 15 ⟩ +≡

```
var (
    chartstack [maxcodes][4 * maxcodes]step    // first와 second 밭만 쓴다
    tmpchart [4 * maxcodes]step
    stk [maxcodes * maxcodes]int    // 아직 다루지 않은 부분나무들의 스택
)
```

67. 함수 *rnbpmJS*는 뿌리 변이 r인 $chartstack[s]$에 대해 Jacquard–Schaeffer RNBPM을 짓는다. 셋째 매개변수 h는 보조 스택 *stk*의 지금 높이다.

$stk[h]$의 값은 $chartstack[s]$에 상태표가 담긴 비스듬한 삼진 나무의 뿌리도 함께 가리킨다. (나무에 마디가 하나뿐이면 뿌리 이름이 상태표에 나타나지 않으므로 이 여분의 문맥이 필요하다.)

```
⟨ 함수들 17 ⟩ +≡
  func rnbpmJS(s, r, h int) {
      var i, j, k, l, m, p, q, t, tt, apip, steps int
      ⟨ 계급 0인 첫 마디의 수 m을 알아내어 쌓는다 68 ⟩
      apip = pip(r, 2)  // 블록을 붙일 꼭지
      for m ≠ 0 {
          m--
          t = stk[h + m]
          ⟨ 다음 T* 밑에 깔린 나무 T를 chartstack[s + 1]에 옮긴다 69 ⟩
          ⟨ 감싼 호가 제자리에 있는지 살핀다 70 ⟩
          steps++
          if l < 0 {
              p = pip(t, 1)  // 빈 나무의 RNBPM은 꾸미지 않은 뿌리 변뿐이다
          } else {
              ⟨ chartstack[s + 1]의 RNBPM을 짓는다 71 ⟩
          }
          splice(irot(p), apip)  // 새 RNBPM을 앞 조각에 이어 붙인다
          apip = pip(t, 2)
      }
      splice(pip(r, 0), apip)  // 모두를 고리로 이어 붙인다
  }
```

68.

```
⟨ 계급 0인 첫 마디의 수 m을 알아내어 쌓는다 68 ⟩ ≡
  if chartstack[s][0].second ≠ 0 {
      confusion("no root bud")
  }
  steps = 1
  for m = 1; ; m++ {
      if chartstack[s][steps].second ≠ 0 {
          confusion("non skew")
      }
      steps++
      stk[h + m] = chartstack[s][steps].second
      steps++
      if stk[h + m] ≡ 0 {
          break
      }
  }
```

이 코드는 67절에서 쓰인다.

69. 여기 이르면 T^*의 걸음들을 볼 채비가 된 것이다. T는 변 $t = stk[h+m]$에 대응하는 부분나무다. T^*가 변 하나짜리 시시한 RNBPM이면 $l = -1$로 두고, 아니면 l을 T^*에서 계급이 0인 마디의 수로 둔다(그것은 계급이 -1인 싹의 수이기도 하다).

둘째 경우에 부분 상태표 T^*는 쉽게 가려낼 수 있다. t에서 tt로 내려가는 걸음으로 시작해서 tt에서 t로 내려가는 걸음으로 끝나기 때문이다. (이 내려가는 걸음은 처음에는 계급 1에서 0으로, 다음에는 3에서 2로 일어난다. 그래서 아래따옴표로 열고 위따옴표로 닫는 독일식 인용부호가 떠오른다고 커누스가 적었다.) 그 두 걸음을 지우고 나머지를 고리처럼 뒤로 밀면 T^*에서 T가 나온다.

⟨ 다음 T^* 밑에 깔린 나무 T를 $chartstack[s+1]$에 옮긴다 69 ⟩ ≡

```
if chartstack[s][steps].second ≡ 0 {
    l = −1
} else {
    tt = chartstack[s][steps].second
    if chartstack[s][steps].first ≠ t {
        confusion("wrong␣parent")
    }
    steps++
    tmpchart[0].first, tmpchart[0].second = 0, 0    // 허수아비 싹
    q, l = 0, 0
    for j = 1; chartstack[s][steps].second ≠ t; steps, j = steps + 1, j + 1 {
        tmpchart[j] = chartstack[s][steps]
        if q ≡ 2 {
            k = j    // 계급 2인 마지막 싹의 자리를 적어 둔다
        } else if q ≡ −1 {
            l++    // 계급 −1인 싹을 센다
        }
        if tmpchart[j].second ≠ 0 {
            q−−
        } else {
            q++
        }    // q가 계급이다
    }
    if chartstack[s][steps].first ≠ tt {
        confusion("right␣bracket")
    }
    for i = k; i < j; i++ {
        chartstack[s + 1][i − k] = tmpchart[i]
    }
    for i = 0; i < k; i++ {
        chartstack[s + 1][i + j − k] = tmpchart[i]
    }
}
steps++
```

이 코드는 67절에서 쓰인다.

70. ⟨ 감싼 호가 제자리에 있는지 살핀다 70 ⟩ ≡

```
if m ≠ 0 ∧ l ≥ 0 ∧
    (chartstack[s][steps].first ≠ t ∨
        chartstack[s][steps].second ≠ stk[h + m − 1]) {
    confusion("arc␣bracketing")
}
```

이 코드는 67 절에서 쓰인다.

71. 커누스는 여기에 "이 걸음에는 더 나은 방법이 있다. 상태표를 옮기면서 꼭지 p 를 곧바로 알아낼 수 있기 때문이다. 그런데 멈춰 서서 그것을 알아낼 틈이 없었다"고 적어 두었다.

⟨ $chartstack[s+1]$ 의 RNBPM을 짓는다 71 ⟩ ≡

```
if chartstack[s + 1][2].second ≠ 0 {
    stk[h + m] = chartstack[s + 1][2].first
} else if chartstack[s + 1][3].second ≠ 0 {
    stk[h + m] = chartstack[s + 1][3].first
} else {
    stk[h + m] = tt
}
rnbpmJS(s + 1, t, h + m)
for p = alphainv[pip(t, 3)]; l ≠ 0; l−− {
    p = alphainv[p]
}
```

이 코드는 67 절에서 쓰인다.

72. 이제 $rnbpmJS$ 가 끝났다. 우리가 눈여겨보는 비스듬한 삼진 나무 넷에 이것을 어떻게 쓰는지 보자.

⟨ 대응하는 평면 지도를 찾아 내놓는다 72 ⟩ ≡

```
for j = 0; j < 4; j++ {
    fmt.Printf("---␣T%s 의␣JS␣지도␣---\n", strings.Repeat("+", j))
    ⟨ 초기 치환을 만든다 51 ⟩
    ⟨ j 번째 비스듬한 나무의 상태표를 chartstack[0] 에 옮긴다 73 ⟩
    stk[0] = inputbud[stack[j + offset − 2]].parent
    rnbpmJS(0, '*', 0)
    printAlpha()
}
```

이 코드는 85 절에도 정의되어 있다.

이 코드는 11 절에서 쓰인다.

73. 상태표를 그 싹의 걸음에서 시작하도록 고리처럼 돌려 옮긴다. 아래 DDP 장에서도 같은 일을 하므로 이름 있는 절로 두었다.

⟨ j 번째 비스듬한 나무의 상태표를 $chartstack[0]$에 옮긴다 73 ⟩ ≡

```
k = 0
for i = inputbud[stack[j + offset − 2]].stepno; i < 4 ∗ n; i, k = i + 1, k + 1 {
    chartstack[0][k] = chart[i]
}
for i = 0; i < inputbud[stack[j + offset − 2]].stepno; i, k = i + 1, k + 1 {
    chartstack[0][k] = chart[i]
}
```

이 코드는 72, 85 절에서 쓰인다.

74. 커누스는 이 대응에 대해 심각한 실망을 적어 두었다. 작은 RNBPM의 바깥 변 l개를 큰 RNBPM 안에 남기는 "T^* 방법"이, 실은 뿌리 다음에 오는 변이 아니라 **마지막** l개를 남기기 때문이다. 그래서 마디와 변의 대응이 몹시 야릇해지고, 그래프 구조를 이해하는 데 이렇다 할 뜻이 없어 보인다.

보기를 들어 (†)에 대응하는 나무는 아주 엉뚱한 꼴이 된다.

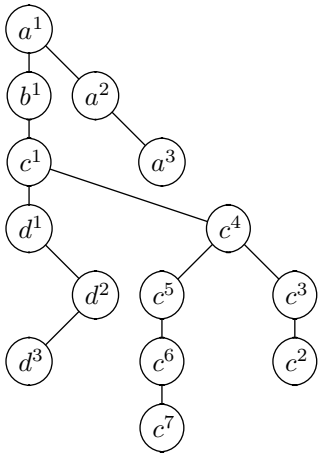


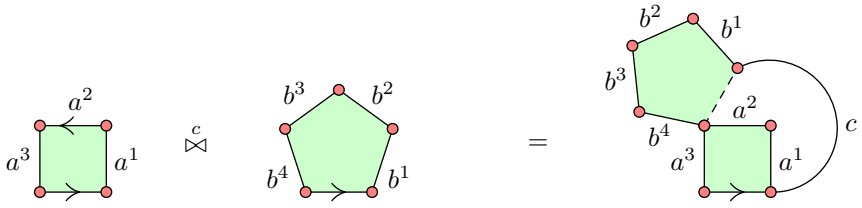
그림 8: 그림 6의 RNBPM에 Jacquard–Schaeffer 대응을 매겨 얻은 비스듬한 삼진 나무.

계급이 0인 마디가 a^1, b^1, c^1, d^1, d^3, c^5, c^6, c^7 이다! (†)의 뿌리 아닌 바깥 변과 수는 같지만, 좋게 말할 것이 그것뿐이다.

고칠 수 없는 문제로 보인다. A^*의 계급 0인 마디들이 마디 a^1에서 멀찍이 떨어져 있기 때문이다.

75. Del Lungo 들의 평면 지도. Del Lungo와 Del Ristoro와 Penaud의 논문은 RNBPM에 비스듬한 삼진 나무를 매기는 전혀 다른 길을 내놓았다. 바탕이 되는 되돌기 분해부터가 다르다. 이것을 DDP 대응이라 부르기로 한다.

(†)에서처럼 RNBPM 하나를 다른 두 겹 이름표 RNBPM m개로 짓는 대신, DDP 대응은 '$\overset{c}{\bowtie}$'라는 흥미로운 **이항** 연산에 기댄다. 그것은 RNBPM 둘로 RNBPM 하나를 만드는데, 하나(S)는 두 겹 뿌리이고 다른 하나(T)는 홑 뿌리다. 다음 그림이 그 연산을 보여 준다.



(‡)

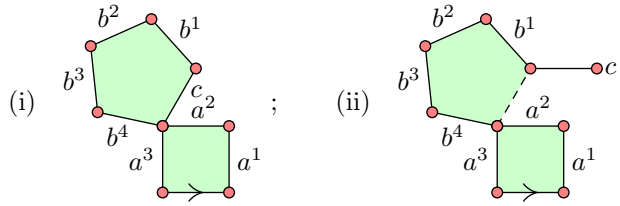
그림 9: 연산 $S \overset{c}{\bowtie} T$. 초록 음영은 속이 복잡할 수 있음을 뜻한다.

여기서 S는 뿌리 변 말고도 바깥 면에 변 a^1, a^2, a^3이 있고, 변 a^2가 둘째 뿌리다(첫 뿌리 변에는 이름이 없다는 것으로 가려낸다). 마찬가지로 T에는 바깥 변 b^1, b^2, b^3, b^4가 있고 홑 뿌리다. T의 뿌리 변이 꼭짓점 u에서 v로 가고 S의 주 뿌리가 s를, 둘째 뿌리가 w를 가리킨다면, 이 연산은 (i) u와 w를 하나로 겹치고, (ii) T의 뿌리 변을 지우고, (iii) s에서 v로 가는 새 변 c를 놓아 두 RNBPM을 붙인다.

76. $\overset{c}{\bowtie}$의 가장 작은 경우들은 따로 챙겨야 한다. T가 뿌리 변뿐이면 s에서 w로 가는 새 변 c를 그냥 더한다. S가 뿌리 변뿐이면 둘째 뿌리가 첫 뿌리를 뒤집은 것이라고 여긴다. 그러면 결국 T의 뿌리를 둘로 쪼개고 그 둘째가 c가 되는 셈이다.

77. 이 구성은 S와 T에 대응하는 quad-edge 치환에서 이어붙이기를 셋 하는 것과 같다. S의 뿌리 변을 r라 하면 이 보기에서 바깥 면은 고리 $(r_3 a_3^3 a_3^2 a_3^1)$이다. 보조 뿌리 변은 a^2이므로 w는 a_0^2을 품은 꼭짓점 고리다. (S가 뿌리 변 r 뿐이라면 바깥 면은 $(r_3 r_1)$이고 w는 (r_2)다.) T의 뿌리 변을 c라 하면 바깥 면 고리는 $(c_3 b_3^4 b_3^3 b_3^2 b_3^1)$이고 $u = (c_2 \dots)$, $v = (c_0 \dots)$다.

걸음 (i)은 $(c_2 a_2^3)$로 이어붙이는 것에 해당한다. 이것이 S를 T에 붙인다. 걸음 (ii)는 $(c_2 x)$로 이어붙이는 것인데, 여기서 $x = c_2\alpha$는 $u = w$의 고리에서 c로부터 반시계로 첫 꼭지다. 이러면 변 c가 "대롱대롱" 남는다.



마지막으로 $(c_2 a_2^1)$로 이어붙이면 $S \overset{c}{\bowtie} T$가 나온다.

78. 변 이름을 매기는 차례가 달라졌으므로 표준 치환도 달라진다. 손쉽게 견줄 수 있도록, 뿌리 아닌 변이 셋 이하인 경우를 다시 적어 둔다. 지금부터 지을 비스듬한 삼진 나무도 함께 보인다.

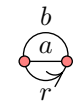


$(r_0)(r_2)(r_3r_1)$

$\emptyset$



$(r_0a_2)(a_0r_2)(r_3a_3)(a_1r_1)$



$(r_0b_2a_2)(b_0r_2a_0)(r_3b_3)(b_1a_3)(a_1r_1)$



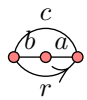
$(r_0a_2)(a_0b_2)(b_0r_2)(r_3b_3a_3)(a_1b_1r_1)$



$(r_0c_2b_2a_2)(r_2a_0b_0c_0)(r_3c_3)(c_1b_3)(b_1a_3)(a_1r_1)$



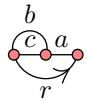
$(r_0b_2a_2)(r_2a_0c_0)(b_0c_2)(r_3c_3b_3)(r_1a_1)(b_1c_1a_3)$



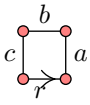
$(r_0c_2a_2)(r_2b_0c_0)(a_0b_2)(r_3c_3)(r_1a_1b_1)(c_1b_3a_3)$



$(r_0c_2a_2)(c_0b_2a_0)(b_0r_2)(r_3b_3c_3)(c_1a_3)(a_1b_1r_1)$



$(r_0a_2)(a_0b_2c_2)(r_2c_0b_0)(r_3b_3a_3)(b_1c_3)(a_1c_1r_1)$

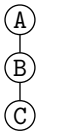


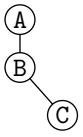
$(r_0a_2)(a_0b_2)(b_0c_2)(c_0r_2)(r_3c_3b_3a_3)(r_1a_1b_1c_1)$

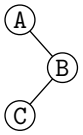




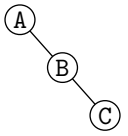


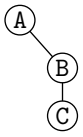












79. RNBPM T와 그 비스듬한 삼진 나무 $\widehat{T}$ 사이의 DDP 대응은 두 가지 중요한 성질을 지니도록 꾸며졌고, 방금 본 보기들에서 둘 다 확인할 수 있다. (1) 위에서 아래로 본 계급 0인 마디들이, 뿌리 꼭짓점에 닿은 뿌리 아닌 변들과 반시계 차례로 대응한다. (2) 앞선 차례로 계급 0인 마지막 마디 뒤에 오는 계급 0인 싹들이, 바깥 고리의 뿌리 아닌 변들과 반시계 차례로 대응한다.

그리고 대응을 정하는 되돌기 규칙이 놀랍도록 간단하다. 가장 간단한 RNBPM(뿌리 변뿐인 것)은 빈 나무에 대응한다(빈 나무에는 계급 0인 싹이 둘 있는데, 성질 (2)에서 뜻이 있는 것은 그중 하나뿐이다). 그렇지 않으면 $S \overset{c}{\bowtie} T$에 대응하는 나무는 (i) $\widehat{S}$와 $\widehat{T}$를 찾고, (ii) 이 프로그램 맨 앞의 고리 돌리기 연산으로 $\widehat{T}^+$를 셈한 다음, (iii) S의 둘째 뿌리에 해당하는 $\widehat{S}$의 계급 0인 싹을, 오른쪽 자식이 $\widehat{T}^+$인 새 마디 c로 바꾸어 얻는다.

이 규칙을 뒤집으려면 이렇게 본다. 나온 비스듬한 삼진 나무에서 S와 T와 c를 되찾을 수 있다. c는 앞선 차례로 계급 0인 마지막 마디다. c의 오른쪽 부분나무를 R라 하면 $R = \widehat{T}^+$이므로 $\widehat{T} = R^-$이고, $\widehat{S}$는 c와 R를 빼서 얻는다. 그리고 S의 둘째 뿌리는 c의 부모 마디에 해당한다.

80. 방금 말한 것을 구현한 함수다. *rnbpmDDP*는 뿌리 변이 r인 *chartstack*$[s]$에 대해 RNBPM을 짓는다. 상태표 뒤에는 특별한 걸음이 하나 따라붙는데, *second* 밭이 *sentinel*이고 *first* 밭이 뿌리의 이름이다.

```
⟨ 함수들 17 ⟩ +≡
  func rnbpmDDP(s, r int) {
      var c, i, j, jj, k, p, q, rr, steps int
      ⟨ 앞선 차례로 계급 0인 마지막 마디 c와 그 부모 p를 찾는다 81 ⟩
      ⟨ c의 오른쪽 부분나무 R에서 R⁻를 chartstack[s+1]에 옮긴다 82 ⟩
      if rr ≠ 0 {
          rnbpmDDP(s+1, c)   // T의 RNBPM을 되돌아 짓는다
      }
      ⟨ 나무의 나머지를 chartstack[s+1]에 옮긴다 83 ⟩
      if p ≠ 0 {
          rnbpmDDP(s+1, r)   // S의 RNBPM을 되돌아 짓는다
      }
      ⟨ 마법 같은 이어붙이기 셋으로 모두를 엮는다 84 ⟩
  }
```

81. 상태표의 걸음은 앞선 차례를 따른다.

⟨ 앞선 차례로 계급 0 인 마지막 마디 c 와 그 부모 p 를 찾는다 81 ⟩ ≡

```
if chartstack[s][0].second ≠ 0 {
    confusion("no root bud")
}
q = −1
for steps = 1; chartstack[s][steps].second ≠ sentinel; steps++ {
    if q ≡ −1 {
        j = steps
    }
    if chartstack[s][steps].second ≡ 0 {
        q++
    } else {
        q−−
    }
}
if q ≠ 2 {
    confusion("bad rank at end")
}
c = chartstack[s][j − 1].second
if c ≡ 0 {   // c 가 상태표에 담긴 나무의 뿌리다
    if j ≠ 1 {
        confusion("parentless rank -1 bud not at beginning")
    }
    c, p = chartstack[s][steps].first, 0
} else {
    p = chartstack[s][j − 1].first
}
if chartstack[s][j + 1].second ≠ 0 {
    confusion("not the last zero")
}
```

이 코드는 80 절에서 쓰인다.

82. c의 오른쪽 자식이 싹뿐이면 부분나무 R가 비어 있고, 그것은 빈 나무에 대응한다. 그렇지 않으면 R는 상태표에서 c로부터 그 뿌리 마디로 갔다 돌아오는 호에 감싸여 있다. *rnbpmJS* 에서 부분나무 T^*가 감싸여 있던 것과 같다. 이번에는 0을 셀 것이 없으므로 옮기는 일이 더 간단하다.

⟨ c의 오른쪽 부분나무 R에서 R^-를 $chartstack[s+1]$에 옮긴다 82 ⟩ ≡

```
jj, steps = j − 1, j + 2
rr = chartstack[s][steps].second
if rr ≠ 0 {   // rr 는 비어 있지 않은 부분나무 R의 뿌리다
    if chartstack[s][steps].first ≠ c {
        confusion("wrong␣parent")
    }
    steps++
    tmpchart[0].first, tmpchart[0].second = 0, 0   // 허수아비 싹
    q = 0
    for j = 1; chartstack[s][steps].second ≠ c; steps, j = steps + 1, j + 1 {
        tmpchart[j] = chartstack[s][steps]
        if q ≡ 2 {
            k = j   // 계급 2인 마지막 싹의 자리를 적어 둔다
        }
        if tmpchart[j].second ≠ 0 {
            q−−
        } else {
            q++
        }   // q가 계급이다
    }
    if chartstack[s][steps].first ≠ rr {
        confusion("right␣bracket")
    }
    for i = k; i < j; i++ {
        chartstack[s + 1][i − k] = tmpchart[i]   // R⁻ 로 밀어 옮긴다
    }
    for i = 0; i < k; i++ {
        chartstack[s + 1][i + j − k] = tmpchart[i]
    }
    chartstack[s + 1][j].second = sentinel
    if chartstack[s + 1][2].second ≠ 0 {
        chartstack[s + 1][j].first = chartstack[s + 1][2].first
    } else if chartstack[s + 1][3].second ≠ 0 {
        chartstack[s + 1][j].first = chartstack[s + 1][3].first
    } else {
        chartstack[s + 1][j].first = rr
    }
}
steps++
```

이 코드는 80 절에서 쓰인다.

83. c가 나무의 뿌리이면 p가 0이고 부분나무 $\widehat{S}$가 비어 있어 할 일이 없다. 그렇지 않으면 $\widehat{S}$에 대응하는 나무는 c와 R를 그냥 빼서 얻는다. 그때 jj는 p에서 c로 가는 호를, $steps$는 c에서 p로 돌아오는 호를 가리킨다.

⟨ 나무의 나머지를 $chartstack[s+1]$에 옮긴다 83 ⟩ ≡

```
if p ≠ 0 {
    for i = 0; i < jj; i++ {
        chartstack[s+1][i] = chartstack[s][i]
    }
    chartstack[s+1][i].first, chartstack[s+1][i].second = 0, 0   // c 자리의 싹
    i++
    for steps++; ; steps, i = steps+1, i+1 {
        chartstack[s+1][i] = chartstack[s][steps]
        if chartstack[s+1][i].second ≡ sentinel {
            break
        }
    }
}
```

이 코드는 80절에서 쓰인다.

84. 마지막으로 위에서 말한 $\overset{c}{\bowtie}$의 세 걸음 이어붙이기를 따른다. 시시한 경우에는 좀 까다로운 손놀림이 필요하다.

⟨ 마법 같은 이어붙이기 셋으로 모두를 엮는다 84 ⟩ ≡

```
if rr ≡ 0 {   // T̂가 비어 있다
    if p ≠ 0 {
        splice(pip(c,0), alpha[pip(p,0)])
    } else {
        splice(pip(c,0), pip(r,2))
    }
} else {
    if p ≠ 0 {
        splice(pip(c,2), alpha[pip(p,0)])
    } else {
        splice(pip(c,2), pip(r,2))
    }
    splice(pip(c,2), alpha[pip(c,2)])
    verts += 2   // 같은 꼭짓점의 꼭지 둘을 갈라놓았으므로
}
splice(pip(c,2), irot(alphainv[pip(r,3)]))
```

이 코드는 80절에서 쓰인다.

85. 이제 *rnbpmDDP* 가 끝났다. 비스듬한 삼진 나무 넷에 이것을 쓰는 법은 앞과 같다.

```
⟨ 대응하는 평면 지도를 찾아 내놓는다 72 ⟩ +≡
  for j = 0; j < 4; j++ {
    fmt.Printf("---␣T%s 의␣DDP␣지도␣---\n", strings.Repeat("+", j))
    ⟨ 초기 치환을 만든다 51 ⟩
    ⟨ j 번째 비스듬한 나무의 상태표를 chartstack[0] 에 옮긴다 73 ⟩
    chartstack[0][k].first = inputbud[stack[j + offset − 2]].parent
    chartstack[0][k].second = sentinel
    rnbpmDDP(0, '*')
    printAlpha()
  }
```

86. 보기를 들어 이 프로그램 맨 앞의 비스듬한 삼진 나무 T 에 DDP 대응을 매기면 이 RNBPM 이 나온다.

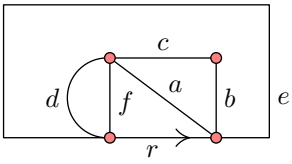


그림 10: 그림 1 의 나무 T 에 대응하는 RNBPM.

α 치환은

$$(e_2b_2a_2r_0)(f_0d_0e_0r_2)(c_0d_2f_2a_0)(c_2b_0)(a_1f_1r_1)(e_3r_3)(b_1c_1a_3)(e_1d_3c_3b_3)(f_3d_1)$$

이다.

87. 이 구성에서 짝수 계급 마디의 수는 안쪽 면의 수와 같고, 홀수 계급 마디의 수는 꼭짓점의 수에서 2 를 뺀 것과 같음을 밝힐 수 있다.

(사실 꼭짓점 마디와 면 마디의 실제 계급은, 아래에서 말할 결과 덕분에 RNBPM 을 알맞은 깊이 우선 차례로 훑으면 "읽어 낼" 수 있다.)

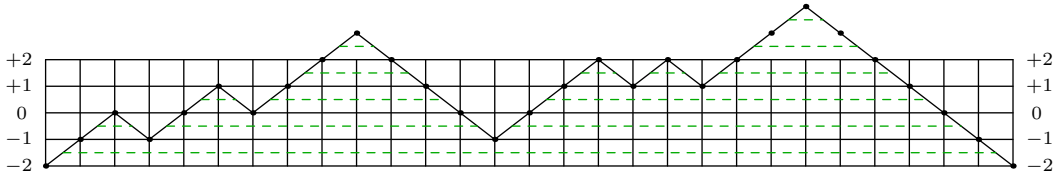
88. 그런데 이 프로그램이 커다란 놀라움으로 이어졌다. 훨씬, 훨씬 더 많은 것이 참이기 때문이다. 커누스는 DDP 대응을 처음 손으로 살펴보던 모든 경우에, T, T^+, T^{++}, T^{+++} 에서 얻은 RNBPM 넷이 서로 쌍대 그래프임을 알아챘다. 손으로는 미덥게 다루기 어려울 만큼 큰 보기에서도 그 짐작을 확인하려고 이 프로그램을 지었는데, 큰 무작위 보기에서도 언제나 맞았다.

좀 더 또렷이 말하면, 켤레인 비스듬한 삼진 나무 넷에 DDP 대응을 매겨 얻은 꼭지 치환 넷을 α_0, α_1, α_2, α_3 이라 할 때, 셈해 본 모든 경우에 $\alpha_k = \hat\rho^k\alpha_0\hat\rho^{-k}$ 였다(치환의 곱은 왼쪽 것부터 적용한다). 여기서 $\hat\rho$ 는 치환 $(r_1r_3)\rho(r_1r_3)$ 다. ρ 와 비슷하되 r 의 아래 첨자는 **내리고** 다른 변의 아래 첨자는 (4 를 법으로) 올린다. 그러니 α_k 는 α_0 의 꼭지 이름을 바꾸기만 한 것이다. α_0 의 순환마다 r 이 아닌 변의 아래 첨자를 k 만큼 내리고 r 의 아래 첨자는 k 만큼 올리면 된다. 보기를 들어 위의 $(e_2b_2a_2r_0)$ 은 α_1 에서 $(e_1b_1a_1r_1)$ 이 된다. k 가 홀수이면 짝수 꼭지가 홀수 꼭지로 바뀌므로 꼭짓점 순환이 면 순환이 되고, 그래서 넷이 번갈아 쌍대가 된다.

89. 증거가 워낙 압도적이라 커누스는 여러 전문가에게 도움을 청했다. 그런데 뜻밖의 행운으로 꼭 알맞은 사람에게 물었으니, 바로 Gilles Schaeffer 였다. 그는 앞에 인용한 Jacquard 와의 논문을 쓴 뒤로도 나무와 평면 지도의 관계를 계속 연구했다고 답했다. 그 결실이 그의 박사 논문 *Conjugaison d'arbres et cartes combinatoires aléatoires*(l'Université Bordeaux I, 1998)다. 커누스는 그 논문을 내려받아 보고는, RNBPM 말고도 여러 주제를 아우르는 깊고 새로운 결과가 가득한 놀라운 보고임을 알았다(그중 많은 것을 그는 따로 발표하지 않기로 했다고 한다). 특히 65–67 쪽에 그는 삼진 나무와 RNBPM 사이의 $(2n+2)$ 대 4 대응을 스케치해 두었는데, 그것이 바로 DDP 대응을 따로 발견한 것이었다. 다만 비스듬한 삼진 나무와의 관계를 대놓고 말하지는 않았다.

90. 그 세 쪽에 담긴 Schaeffer 의 놀라운 구성이 모든 것을 설명해 준다. 그것으로 앨리스가 나무 둘레를 기어가는 동안 대응하는 평면 그래프를 "실시간으로" 지을 수 있음을 밝힐 수 있다.

곧, 상태표에 내려가는 걸음 넷을 더하고 걸음에 새 이름표를 매기면 된다(들어가며에 나온 비스듬한 삼진 나무로 보이면 이렇다).



$a_2\ a_3\ b_2\ b_3\ b_0\ c_3\ c_0\ c_1\ c_2\ b_1\ a_0\ d_3\ e_2\ e_3\ e_0\ e_1\ d_0\ d_1\ f_0\ f_1\ f_2\ f_3\ d_2\ a_1\ r_2\ r_3\ r_0\ r_1$

그림 11: 같은 상태표에 Schaeffer 의 이름표를 매긴 것. 이름표 $4n+4$ 개가 치환의 꼭지 하나씩에 대응한다.

올라가는 걸음의 이름표는 x_i 인데, i 는 걸음을 시작할 때의 계급(4 를 법으로)이고 x 는 이 싹에 붙은 마디의 이름이다. 내려가는 걸음의 이름표는 y_j 인데, j 는 걸음을 시작할 때의 계급에 2 를 더한 것(4 를 법으로)이고 y 는 내려가서 닿는 마디의 이름이다. 마지막 걸음 넷의 이름표는 r_2, r_3, r_0, r_1 이다. 이렇게 이름표 $4n+4$ 개를 매기면 치환의 꼭지마다 하나씩 돌아가고, 꼭지들은 짝 $\{x_i, y_j\}$ 으로 맞물린다. 그런 짝은 모두 α 가 $x_i \mapsto y_{j-1}$ 로, $y_j \mapsto x_{i-1}$ 로 보낸다는 뜻이다(아래 첨자는 4 를 법으로).

이 규칙이 옳다는 것은 귀납법으로 쉽게 밝혀진다. 아주 강한 귀납 가정이기 때문이다. 그리고 쌍대성이 치환 α_k 에 어떻게 미치는가에 대한 커누스의 짐작이 그 바로 따름정리다.

91. 이제 끝났다. 딱 하나, 부디 한 번도 불리지 않기를 바라는 함수만 남았다.

```
⟨ 함수들 17 ⟩ +≡
  func confusion(id string) {  // 단언이 깨졌다
    fmt.Fprintf(os.Stderr, "이런␣일은␣있을␣수␣없다␣(%s)!\n", id)
    os.Exit(99)
  }
```

92. 돌려 보기. 쓰는 법은 이렇다. 마디마다 네 글자짜리 인자를 하나씩 명령줄로 준다.

```
skew-ternary-calc A-BD B--C C--- DE-F E--- F---
```

들어가며에 나온 나무 T가 바로 이것이다. 그러면 먼저 싹까지 드러낸 나무가 나오고,

```
........  A:  3 B  D
........  B:  5  2 C
.........  C:  7  6  8
.........  D:  E  4 F
........  E:  9 11 10
..........  F: 13 12 14
```

이어서 켤레 셋이 나온다.

```
+:  F--- C--- B--C A--B D-FA E--D
++:  F--- C--- B--C A--B D-FA E-D-
+++:  C--- B--C A--B E--- DAE- F--D
```

그 뒤로 비스듬한 나무 넷의 JS 지도 넷과 DDP 지도 넷이 차례로 나온다. 들여쓴 점의 개수가 계급이고(계급 0이 점 여덟 개), 숫자는 싹 번호다.

93. 옮긴 것이 원본과 같은지 커누스의 C 판을 나란히 돌려 맞춰 보았다. 마디가 1개부터 62개까지인 무작위 비스듬한 삼진 나무와, 나무가 아닐 수도 있는 무작위 인자 꾸러미와, 잘못된 인자로 부르는 여러 길을 통틀어 만 번 넘게 돌렸다. 알림말은 한국어로 옮겼으므로 그것만 서로 대응시켜 두고 견주었는데, 나머지는 한 글자도 다르지 않았다.

94. 옮기면서 손댄 곳은 다섯이다. 모두 언어에서 온 것이다.

첫째, 매크로가 함수가 되었다. *pip*, *pipEdge*, *pipSub*, *rot*, *irot*이 그렇다. 그리고 C 원본이 *root*라는 이름으로 가리키던 *inputbud*[1].*parent*는, Go에 그런 별명이 없으므로 그 자리를 그대로 쓴다.

둘째, 돌리기 ρ를 적는 방식을 바꾸었다. 원본은 가지치기를 피하려고 `((p)+1)^(((p)^((p)+1))&-4)`라는 비트 재주를 쓰는데, 여기서는 `p&^3 | (p+1)&3`이라고 적었다. 아래 두 비트만 4를 법으로 올린다는 뜻이 그대로 드러난다. 하는 일은 같다.

셋째, 인자의 자식 셋을 붙이는 대목을 고리 하나로 묶었다. 원본은 왼쪽, 가운데, 오른쪽을 펼쳐 적었는데 그래야 오류 부호가 저마다 달라 어느 자식에서 걸렸는지 알 수 있기 때문이다. 그것은 자식 번호를 부호에 얹어 그대로 살렸다.

넷째, 쓰이지 않는 변수를 뺐다. 원본은 *curbud*라는 전역 변수와, *rnbpm_ddp* 안의 t와 *parent*를 밝혀 두지만 어디서도 쓰지 않는다. Go에서는 쓰이지 않는 지역 변수가 컴파일 오류다.

다섯째, 그만둘 때의 종료 부호를 양수로 바꾸었다. 원본은 *exit*(-666)처럼 음수를 쓰지만 Go에서는 0과 125 사이를 권한다. 커누스의 부호 -10부터 -66까지는 부호만 뒤집었고, -666만 99로 바꾸었다.

95. 본문에서도 원본과 다르게 적은 곳이 셋 있다. 모두 원본의 잘못을 바로잡은 것이다.

첫째, 켤레 넷의 지도 사이의 등식을 원본은 $\alpha_k = \alpha_0\hat{\rho}^k$ 라고 적었다. 그런데 α 는 짝수 꼭지를 짝수 꼭지로 보내고 $\hat{\rho}$ 는 홀짝을 뒤집으므로, 곱 $\alpha_0\hat{\rho}$ 는 짝수 꼭지를 홀수 꼭지로 보낸다. 그런 치환은 어떤 지도의 α 도 될 수 없다. 보기를 들어 $\alpha_0\hat{\rho}$ 는 $e_2 \mapsto b_3$ 인데 α_1 은 $e_2 \mapsto r_0$ 이다. 커누스가 함께 적어 둔 α_1, α_2, α_3 은 이 프로그램의 출력과 순환 하나하나까지 같고, 그것들은 모두 이름을 바꾼 것, 곧 켤레 $\hat{\rho}^k\alpha_0\hat{\rho}^{-k}$ 다. 그래서 그렇게 적었다. 앱의 `renamesTo` 가 무작위 나무마다 셈해 보는 것도 이 꼴이다.

둘째, Schaeffer 의 박사 논문 제목을 원본은 *Conjugation d'arbres et cartes combinatoires aléatores* 라고 적었다. 그의 문헌 목록에 따라 *Conjugaison* 과 *aléatoires* 로 고쳤다.

셋째, 초기 치환을 만드는 대목에서 원본은 마디에 "가운데 자식과 뿌리 자식"도 있다고 적었는데, 뿌리가 아니라 오른쪽 자식을 말한 것이다.

96. 그림도 커누스가 함께 내놓은 `skew-ternary-calc.mp` 의 것을 그대로 쓰지만, 부르는 방식이 달라서 손을 좀 보아야 했다. 원본은 `mpost` 가 파일을 처음부터 끝까지 한 번에 훑는 것을 전제로 한다. 그래서 그림들이 앞 그림이 남긴 상태를 물려받는다. 우리는 `luamplib` 으로 조판 중에 그림을 그리는데, 그것은 그림마다 `.mp` 를 다시 읽되 부른 `def` 하나만 실행한다. 그 물림이 끊기므로, 물려받던 것을 모두 드러내어 그림마다 홀로 서게 했다.

드러내야 했던 것이 넷이다. 하나, 그림 1 과 그림 41 이 만들던 이름표 그림(마디 이름 `A...N` 과 계급 숫자)이 뒤의 그림들로 흘러 들어갔다. 그것을 앞으로 끌어올렸다. 둘, 그림 사이에서 `brangle` 과 `h` 와 `v` 가 바뀐다. 그림마다 제 값을 앞세우게 했다. 셋, 그림 3 은 그림 2 의 앞부분을 `saveit` 에 담아 두었다 쓰고, 그림 5 는 그림 4 에 대해 같은 일을 한다. 그 앞부분을 `def` 로 갈라내어 둘이 나눠 쓰게 했다. 넷, 그림 50 이 제 안에 두던 `setup` 을 그림 51 과 52 가 쓴다. 그것도 밖으로 내었다.

그 밖에 `luamplib` 이 `verbatimtex` 를 거치지 않으므로 거기 있던 `\matname` 과 `\budname` 을 문서 림보로 옮겼고, 이름표에 쓰인 `cwebmac` 의 세로줄 매크로는 `GWEB` 에서 다른 뜻이라 한 글자 수식으로 바꾸었다. 그림은 예순여덟 장 모두 원본과 같은 모습으로 나온다.

97. 색인.

abort0: 17, 20, 24, 27.
abort1: 17, 18.
abort2: 17, 18, 19.
alpha: 50, 51, 52, 54, 55, 84.
alphainv: 50, 51, 52, 71, 84.
apip: 67.
arg: 18, 19.
Args: 12, 17, 18.
b: 33.
branch: 31, 34.
bud: 14, 15.
buds: 15, 22, 24, 26, 27.
budstep: 31, 33.
c: 16, 17, 18, 19, 53, 80, 81, 82, 83, 84.
chart: 29, 33, 34, 35, 36, 37, 73.
chartstack: 66, 67, 68, 69, 70, 71, 73, 80, 81, 82, 83, 85.
code: 17.
confusion: 22, 23, 33, 34, 35, 37, 51, 52, 53, 68, 69, 70, 81, 82, 91.
createsteps: 31, 34, 35.
curbud: 94.
d: 21, 23, 24.
downward: 23, 24.
downwardmid: 22, 24.
Exit: 12, 17, 20, 25, 91.
exit: 94.
f: 53, 55.
fillbuds: 20, 25, 26, 31.
first: 28, 33, 34, 35, 37, 66, 69, 70, 71, 80, 81, 82, 83, 85.
fmt: 12, 17, 20, 25, 36, 37, 38, 39, 53, 54, 55, 72, 85, 91.
Fprintf: 12, 17, 20, 25, 91.
h: 67, 68, 69, 70, 71.
i: 16, 19, 49, 67, 69, 73, 80, 82, 83.
id: 91.
inputbud: 15, 20, 22, 24, 26, 33, 35, 37, 38, 72, 73, 85, 94.
inputnode: 15, 18, 19, 20, 23, 24, 25, 26, 31, 34, 39, 51.
irot: 49, 67, 84, 94.
j: 16, 17, 18, 19, 20, 37, 67, 69, 72, 73, 80, 81, 82, 85.
jj: 80, 82, 83.
k: 15, 16, 37, 51, 67, 69, 73, 80, 82, 85.
l: 36, 67, 69, 70, 71, 74.
left: 13, 14, 19, 23, 24, 26, 31, 39, 51.
len: 12, 18.
m: 36, 67, 68, 69, 70, 71.
main: 11.
match: 28, 34.
maxcodes: 13, 29, 66.
message: 17.
middle: 13, 14, 19, 23, 24, 26, 31, 39.
n: 15, 18, 27, 35, 36, 37, 51, 53, 54, 55, 66, 73.
node: 14, 15.
offset: 32, 33, 34, 35, 37, 72, 73, 85.
os: 12, 17, 18, 20, 25, 91.
p: 16, 19, 21, 22, 23, 24, 25, 26, 31, 34, 36, 39, 49, 52, 54, 55, 67, 71, 80, 81, 83, 84.
parent: 13, 14, 19, 20, 22, 23, 24, 35, 37, 38, 72, 85, 94.
pip: 49, 51, 54, 55, 67, 71, 84, 94.
pipEdge: 49, 54, 55, 94.
pipSub: 49, 54, 55, 94.
Print: 37, 39.
printAlpha: 53, 72, 85.
Printf: 36, 39, 53, 54, 55, 72, 85.
printfam: 36, 37.
Println: 37, 38, 54, 55.
printTree: 38, 39.
q: 21, 22, 23, 24, 31, 34, 36, 52, 54, 55, 67, 69, 80, 81, 82.
r: 25, 26, 36, 52, 54, 55, 67, 80, 84.
rank: 14, 18, 20, 25, 26, 28, 33, 34, 35, 39.
Repeat: 37, 39, 72, 85.
right: 13, 14, 19, 23, 24, 26, 31, 39.
rnbpm_ddp: 94.
rnbpmDDP: 80, 85.
rnbpmJS: 67, 71, 72, 82.
root: 20, 94.
rot: 49, 52, 94.
rr: 80, 82, 84.
s: 52, 67, 68, 69, 70, 71, 80, 81, 82, 83.
second: 28, 34, 36, 37, 66, 68, 69, 70, 71, 80, 81, 82, 83, 85.
sentinel: 13, 14, 18, 37, 80, 81, 82, 83, 85.
setmate: 20, 21, 26.
splice: 52, 67, 84.
stack: 29, 33, 34, 35, 37, 72, 73, 85.
stacked: 29, 32, 33, 34, 35.
Stderr: 12, 17, 20, 25, 91.
step: 28, 29, 66.

stepno: 14, 33, 37, 73.
steps: 29, 33, 34, 35, 36, 37, 67, 68, 69, 70, 80, 81, 82, 83.
stk: 66, 67, 68, 69, 70, 71, 72.
strings: 37, 39, 72, 85.
t: 54, 55, 67, 69, 70, 71, 94.
tmpchart: 66, 69, 82.
tt: 67, 69, 71.
u: 49.
upward: 23.
v: 53, 54.
verts: 50, 51, 52, 53, 84.
x: 20.

⟨ c의 오른쪽 부분나무 R에서 R^-를 $chartstack[s+1]$에 옮긴다 82 ⟩ 80 절에서 쓰임
⟨ T의 켤레 셋을 찾아 내놓는다 30 ⟩ 11 절에서 쓰임
⟨ $chartstack[s+1]$의 RNBPM을 짓는다 71 ⟩ 67 절에서 쓰임
⟨ j번째 비스듬한 나무의 상태표를 $chartstack[0]$에 옮긴다 73 ⟩ 72, 85 절에서 쓰임
⟨ 가운데를 따라 내려가 짝을 놓는다 24 ⟩ 21 절에서 쓰임
⟨ 가운데를 따라 위로 올라가 옆으로 건너뛴다 23 ⟩ 21 절에서 쓰임
⟨ 감싼 호가 제자리에 있는지 살핀다 70 ⟩ 67 절에서 쓰임
⟨ 계급 0인 첫 마디의 수 m을 알아내어 쌓는다 68 ⟩ 67 절에서 쓰임
⟨ 꼭짓점 고리를 찍고 센다 54 ⟩ 53 절에서 쓰임
⟨ 나무를 온전히 채웠는지 살핀다 27 ⟩ 20 절에서 쓰임
⟨ 나무의 나머지를 $chartstack[s+1]$에 옮긴다 83 ⟩ 80 절에서 쓰임
⟨ 다음 T^* 밑에 깔린 나무 T를 $chartstack[s+1]$에 옮긴다 69 ⟩ 67 절에서 쓰임
⟨ 대응하는 평면 지도를 찾아 내놓는다 72 ⟩ 11 절에서 쓰임
⟨ 마법 같은 이어붙이기 셋으로 모두를 엮는다 84 ⟩ 80 절에서 쓰임
⟨ 면 고리를 찍고 센다 55 ⟩ 53 절에서 쓰임
⟨ 명령줄을 처리한다 12 ⟩ 11 절에서 쓰임
⟨ 상수 13 ⟩ 11 절에서 쓰임
⟨ 상태표를 만든다 35 ⟩ 30 절에서 쓰임
⟨ 상태표에서 켤레들을 내놓는다 37 ⟩ 30 절에서 쓰임
⟨ 세 자식을 차례로 훑으며 빈자리에 싹을 넣는다 26 ⟩ 25 절에서 쓰임
⟨ 싹 둘을 새로 마련한다 22 ⟩ 21 절에서 쓰임
⟨ 싹까지 모두 드러낸 나무를 내놓는다 38 ⟩ 30 절에서 쓰임
⟨ 싹을 넣고 계급을 셈한다 20 ⟩ 18 절에서 쓰임
⟨ 앞선 차례로 계급 0인 마지막 마디 c와 그 부모 p를 찾는다 81 ⟩ 80 절에서 쓰임
⟨ 인자를 새긴다. 비스듬한 나무가 아니면 알리고 그만둔다 18 ⟩ 12 절에서 쓰임
⟨ 자료형 14 ⟩ 11 절에서 쓰임
⟨ 자식 셋을 붙인다 19 ⟩ 18 절에서 쓰임
⟨ 전역 변수 15 ⟩ 11 절에서 쓰임
⟨ 지역 변수 16 ⟩ 11 절에서 쓰임
⟨ 초기 치환을 만든다 51 ⟩ 72, 85 절에서 쓰임
⟨ 함수들 17 ⟩ 11 절에서 쓰임

비스듬한 삼진 나무

	절	쪽
들어가며	1	1
새기기	13	7
상태표	28	14
평면 지도를 담는 quad-edge 자료 구조	40	20
평면 그래프의 벽돌	56	27
Jacquard와 Schaeffer의 평면 지도	62	29
Del Lungo 들의 평면 지도	75	35
돌려 보기	92	43
색인	97	45